



JAIN
DEEMED-TO-BE UNIVERSITY

FACULTY OF
ENGINEERING
AND TECHNOLOGY

Department of Aerospace Engineering

Faculty of Engineering and Technology

Jain Global Campus, Kanakapura Taluk - 562112
Ramanagara District, Karnataka, India

A Project Report

On

“OBSTACLE AVOIDANCE FOR UAVs”

Submitted in partial fulfilment for the award of the degree of

BACHELOR OF TECHNOLOGY

IN

AERONAUTICAL ENGINEERING

Submitted by

Akash Bhat 18BTRAN002

Manu A 18BTRAN025

Rohan M 18BTRAN036

Abhishek V 18BTRAN001

Under the guidance of

Dr.K.Natarajan

Professor & Functional head-Avionics

Department of Aerospace Engineering

Faculty of Engineering and Technology

JAIN (Deemed-to-be University)

2021-2022

CERTIFICATE

THIS IS TO CERTIFY THAT THE PROJECT WORK TITLED **OBSTACLE AVOIDANCE FOR UAVs** IS CARRIED OUT BY ROHAN M (18BTRAN036), ABHISHEK V (18BTRAN001), AKASH BHAT (18BTRAN002), MANU A (18BTRAN025), BONAFIDE STUDENTS OF BACHELOR OF TECHNOLOGY, AT THE INTERNATIONAL INSTITUTE FOR AEROSPACE ENGINEERING AND MANAGEMENT, JAIN (DEEMED-TO-BE UNIVERSITY), IN PARTIAL FULFILMENT FOR THE AWARD OF **BACHELOR OF TECHNOLOGY IN AERONAUTICAL ENGINEERING**, DURING THE YEAR **2021-2022**.

Dr.K.Natarajan

Professor & Functional Head-
Avionics, IIAEM,
JAIN (Deemed-to-be University)

Date:

Name of the Examiner

Prof. Antonio Davis

Director and Head, IIAEM,
JAIN (Deemed-to-be University)

Date:

Signature of Examiner

1.

2.

DECLARATION

I, the undersigned students of fourth year B.Tech in Aeronautical Engineering, at the International Institute for Aerospace Engineering And Management, JAIN (Deemed-to-be University), hereby declare that the dissertation titled **OBSTACLE AVOIDANCE FOR UAVs** has been carried out by us and submitted in partial fulfilment for the award of **Bachelor of Technology in Aeronautical Engineering**, during the academic year **2021-2022**. Further, the matter embodied in the project report has not been submitted previously by anybody for the award of any degree or diploma to any University, to the best of our knowledge and faith.

PLACE: BANGALORE

ROHAN M(18BTRAN036)

DATE:

ABHISHEK V(18BTRAN001)

AKASH BHAT(18BTRAN002)

MANU A(18BTRAN025)

ACKNOWLEDGEMENT

It is a great pleasure for us to acknowledge the assistance and support of a large number of individuals who have been responsible for the successful completion of this project work.

First, we take this opportunity to express our sincere gratitude to International Institute of Aerospace Engineering and Management (IIAEM), Jain (Deemed-to-be University) for providing us with a great opportunity to pursue our bachelor's degree.

In particular, we would like to thank Dr. Antonio Davis, Director, International Institute of Aerospace Engineering and Management (IIAEM), Jain University for his constant encouragement and expert advice.

In particular, we would like to thank Dr.K.Natarajan, Professor, International Institute of Aerospace Engineering and Management (IIAEM), Jain University for his constant encouragement and support.

We would like to thank all the lecturers and all the staff members of Aerospace Department for their support and contributions.

We are also grateful to our family and friends who provided us with every requirement throughout the course. We would like to thank one and all who directly or indirectly helped us in completing the Project work successfully.

Place: Bangalore

Date :

Akash Bhat USN: 18BTRAN002

Manu A USN: 18BTRAN025

Rohan M USN: 18BTRAN036

Abhishek V USN: 18BTRAN001

ABSTRACT

Collision avoidance is a key factor in enabling the integration of unmanned aerial vehicle into real life use, whether it is in military or civil application. For a long time there have been a large number of works to address this problem; therefore a comparative summary of them would be desirable. This paper presents a survey on the major collision avoidance systems developed in up to date publications. Each collision avoidance system contains two main parts: sensing and detection, and collision avoidance. Based on their characteristics each part is divided into different categories; and those categories are explained, compared and discussed about advantages and disadvantages in this paper.

Moving towards autonomy, unmanned vehicles rely heavily on state-of-the-art collision avoidance systems (CAS). A lot of work is being done to make the CAS as safe and reliable as possible, necessitating a comparative study of the recent work in this important area. The paper provides a comprehensive review of collision avoidance strategies used for unmanned vehicles, with the main emphasis on unmanned aerial vehicles (UAV). It is an in-depth survey of different collision avoidance techniques that are categorically explained along with a comparative analysis of the considered approaches w.r.t. different scenarios and technical aspects. This also includes a discussion on the use of different types of sensors for collision avoidance in the context of UAVs.

CONTENTS

Declaration	i
Acknowledgement	ii
Abstract	iii
List of figures	iv
Nomenclature	v
CHAPTER 1	1
INTRODUCTION	2
1.1 Types of UAV	2
1.2 Objective	7
1.3 Proposed systems	7
1.4 Methodology	12
CHAPTER 2	18
2.1 Literature survey	19
CHAPTER 3	22
ALGORITHM IMPLEMENTATION	23
3.1 Hardware design	23
3.2 Flow chart for drone obstacle avoidance	27
3.3 Raw code for obstacle avoidance	28
3.4 Implemented code	30
3.5 Hardware testing	38
3.6 Software validation	38
Chapter 4	39
Result and discussion	40
4.1 Discussion	40
4.2 Results	40
Chapter 5	43
Applications	44
Chapter 6	45
Conclusion and future scope	46

Reference

vi

Appendix

vii

LIST OF FIGURES

Fig. No.	Description of the figure	Page No.
Fig 1	Multicopter	3
Fig.2	Glider	4
Fig.3	Single rotor	4
Fig.4	Hybrid VTOL	5
Fig.5	Arduino UNO	6
Fig.6	Ultrasonic sensor	6
Fig.7	F450 frame	7
Fig.8	Brushless motor	8
Fig.9	Electronic speed controller	8
Fig.10	Propellers	9
Fig.11	Battery	9
Fig.12	Transmitter	9
Fig.13	Flight controller	10
Fig.14	Arduino UNO	10
Fig.15	Ultrasonic sensors	11
Fig.16	Jumper cables	11
Fig.17	Drone frame	12
Fig.18	Mounting motors	12
Fig.19	Connection of KK 2.1.5 board	14
Fig.20	Receiver and ESC connection	14
Fig.21	KK 2.1.5 board mounted on to the drone	14
Fig.22	connection between Arduino and sensor	15
Fig.23	Setup in arduino software	16
Fig.24	Setup in Arduino software	16
Fig.25	Assembled drone	17
Fig.26	Assembled drone	17
Fig.27	Assembled drone	17
Fig.28	1000kv Brushless motor design	23
Fig.29	Propeller design	23
Fig.30	Drone arm	23
Fig.31	F450 lower plate	23
Fig.32	F450 upper plate	24
Fig.33	F450 lower plate dimensions	24
Fig.34	F450 upper plate dimensions	25
Fig.35	F450 propeller dimension	25
Fig.36	Motor dimensions	26
Fig.37	Arm dimension	26
Fig.38	Obstacle avoidance code	34
Fig.39	Obstacle avoidance code	35
Fig.40	Obstacle avoidance code	36
Fig.41	Obstacle avoidance code	37
Fig.42	Connections of 4 ultrasonic sensors to Arduino board	38
Fig.43	Output of roll pitch values	41
Fig.44	Output of obstacle avoidance code	42

NOMENCLATURE

UAV	Unmanned aerial vehicle
UAS	unmanned aircraft system
CAS	collision avoidance system
LIDAR	Light detection and ranging
VTOL	Vertical take-off and landing
ESC	Electronic Speed Controller
PWM	Pulse Width Modulation
CW	Clock wise
CCW	Counter clock wise
LIPO	Lithium Ion Polymer
BLDC	Brushless direct current
AFHDS	Automatic Frequency Hopping Digital System
PCB	Printed circuit board
RL	reinforcement learning
DC	Direct current
AC	Alternating current

CHAPTER 1

INTRODUCTION

Unmanned aerial vehicles (UAVs), commonly known as drones, are aircraft that do not have a human pilot on board. UAV flight can be controlled either independently by onboard computers or remotely from the ground or from another pilot vehicle. Autonomous UAVs have been developed in the field, eliminating the requirement for a pilot. These UAVs have an on-board controller that manages the UAV's stability and trajectory movements. Unmanned aerial vehicles are referred to by a variety of titles, all of which pertain to the same concept, such as unmanned aircraft system (UAS).

UAV quad rotor The quad rotor UAV is a small to medium-sized unmanned aerial vehicle with four rotors that lift and propel it. Rotors are symmetrical about the center of gravity and are located at the ends of a cross. Quadcopters typically employ two pairs of identical fixed-pitch or variable-pitch propellers.

The speed of each rotor can be varied to obtain control. The aerodynamic forces operating on the rotorcraft are provided by these rotors, which are modelled using momentum theory and blade element theory. The rotorcraft's expected cargo capacity and lift performance can be calculated using this information.

Unmanned aerial vehicles (UAVs) offer a lot of potential for both civilian and military uses. UAVs can potentially replace manned aerial vehicles in various activities and perform well in curriculums that traditional manned aerial vehicles do not. This is due to their low cost, safety benefit, and mobility.

However, because UAVs are not controlled by humans, they face a number of obstacles to overcome, one of which is collision avoidance. An unmanned aerial vehicle (UAV) must be able to avoid colliding with both static and moving obstacles in order to be useful.

While UAV collision avoidance is fundamentally analogous to that of air traffic and mobile robotics, UAVs have several unique properties that must be taken into account, making them an intriguing research area. There has been a lot of work done in this area. Many publications describing various collision avoidance systems (CASs) have been published during the last two decades.

For both non-autonomous and autonomous, a collision avoidance system is essential. Collisions can occur for a variety of causes, including the operator's/negligence, driver's equipment malfunction, or inclement weather. The degree to which an unmanned vehicle can make decisions without being expressly directed by a remote operator varies depending on the tasks at hand or the degree to which the vehicle can make judgments without being explicitly controlled by a remote operator.

Unmanned vehicles, in general, contain a variety of on-board sensors that can be employed for situational awareness and autonomous decision-making in the field. Overall, control can be manual, based on live video from a vehicle's camera (remote control), or autonomous, depending on feedback from a mounted camera and other types of sensors showing hurdles on their way.

Types of UAVs

An Unmanned Aerial Vehicle (UAV) is a plane that does not have a human pilot on board. In this post, we look at the various types of drones on the market, some of which are only concepts and others of which are now in use.

Drones can be classed in a variety of ways, such as photography drones, aerial mapping drones, surveillance drones, and so on.

The best classification of 'Drones', however, may be formed based on aerial platforms. Drones are classified into four categories based on the type of airborne platform employed.

1. Multi Rotor Drones
2. Fixed Wing Drones
3. Single Rotor Helicopter
4. Fixed Wing Hybrid VTOL

Multi Rotor Drones

Drones with several rotors are the most prevalent, and they are utilised by both professionals and hobbyists. They're employed for a variety of tasks, including aerial photography and video surveillance.

There are a variety of goods available in this market segment, such as multi-rotor drones for commercial usage such as aerial photography, as well as a variety of hobby options such as amateur drone racing or leisure flying.

Multi-rotor drones are the easiest to construct and the cheapest alternative available among the four drone kinds (depending on aerial platform).



Figure 1:- Multicopter

Fixed wing

Fixed-wing drones are not the same as multi-rotor drones in terms of design and construction. They have a wing, just like regular aeroplanes.

Fixed wing drones, unlike multi-rotor drones, never need energy to keep afloat in the air (fixed wing drones can't stand still in the air) and resist gravity. Instead, they continue on their predetermined path or as directed by the guide control (potentially a human-operated remote unit) as long as their energy source allows.



Figure 2:- Gider

The majority of fixed-wing drones have a flight time of a few hours. Drones with gas engines can fly for up to 16 hours. Fixed-wing drones are useful for long-distance operations due to their longer flight length and fuel efficiency (be it mapping or surveillance). They cannot, however, be used for aerial photography because the drone must remain stationary in the air for an extended period of time.

Single rotor

Single-rotor drones resemble helicopters in terms of form and structure. A single rotor drone, unlike a multi-rotor drone, has only one large rotor and a small rotor on the drone's tail to control its direction. Drones with a single rotor are far more efficient than those with several rotors.

They can fly for longer periods of time and are even powered by gas engines. In aerodynamics, the lower the number of rotors, the lower the object's spin. Quadcopters are far more stable than octocopters for this reason. Single-rotor drones are far more efficient than multi-rotor drones in this regard.



Figure 3:- Single rotor

Hybrid VTOL

These are hybrid models that combine the advantages of fixed-wing aircraft (longer flight time) with the advantages of rotor-based models (hover). This notion has been tested since the 1960s,

but with little success. The introduction of new generation sensors (gyros and accelerometers) has given this notion new energy and direction.



Figure 4:- Hybrid VTOL

Arduino UNO

The Arduino UNO is a microcontroller board that uses the ATmega328P microcontroller. It contains 14 digital input/output pins (including 6 PWM outputs), 6 analogue inputs, a 16 MHz ceramic resonator, a USB connection, a power jack, an ICSP header, and a reset button. It comes with everything you'll need to get started with the microcontroller; simply plug it into a computer with a USB cable or power it with an AC-to-DC adapter or battery.

➤ Board Name	Arduino UNO R3
➤ SKU	A000066
➤ Microcontroller	ATmega328P
➤ USB connector	USB-B
➤ Pins Built-in LED Pin	13
➤ Digital I/O Pins	14
➤ Analog input pins	6
➤ PWM pins	6
➤ Communication UART	Yes
➤ I2C	Yes
➤ SPI	Yes
➤ Power I/O Voltage	5V
➤ Input voltage	7-12V
➤ DC Current	20 mA
➤ Power Supply Connector	Barrel Plug
➤ Clock speed Main Processor	ATmega328P 16 MHz
➤ USB-Serial Processor	ATmega16U2 16 MHz
➤ Memory	ATmega328P 2KBSRAM,32KBFLASH,1KBEEPROM
➤ Dimensions	Weight 25 g
➤ Width, length	53.4 mm, 68.6 mm

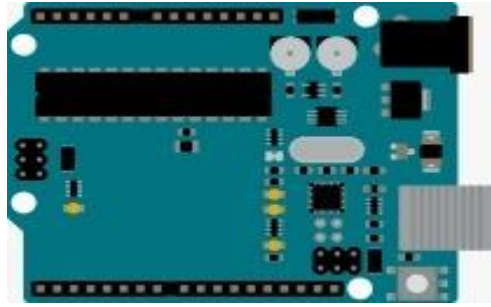


Figure 5:- Arduino UNO

Ultrasonic sensor



Figure 6:- Ultrasonic sensor

An ultrasonic sensor is an electronic device that uses ultrasonic sound waves to detect the distance between a target item and converts the reflected sound into an electrical signal. Ultrasonic waves travel quicker than audible sound waves (i.e. the sound that humans can hear).

The transmitter (which generates sound using piezoelectric crystals) and the receiver are the two primary components of ultrasonic sensors (which encounters the sound after it has travelled to and from the target).

The transmitter (which uses piezoelectric crystals to create sound) and the receiver are the two basic components of ultrasonic sensors (which encounters the sound after it has travelled to and from the target).

The sensor measures the time between the transmitter's sound emission and its contact with the receiver in order to calculate the distance between the sensor and the item. $D = \frac{1}{2} T \times C$ (where D is the distance, T is the time, and C is the speed of sound, which is 343 metres per second) is the formula for this computation.

What may ultrasonic sensors be used for?

The Ultrasonic sensor is used in the following projects. Students can build some of the most recent innovative projects to gain hands-on experience in fields linked to/using the Ultrasonic sensor.

- Mobile Robot Navigation System.
- GPS-Guided Mobile Robot.
- Smart Walking Cane.
- Pneumatic Braking System.
- Automated Solar Grass Cutter.
- Ultrasonic Distance Finder Using 8051.
- Drowsiness Prevention and Collision Avoidance System.
- Collision Avoidance Robot.

Objective

The main objective of this work was to develop a real time obstacle avoidance algorithm to allow future UAV applications, where these vehicles are going to have to navigate sharing space with humans and other objects.

Proposed systems

F450 Frame

It's a high-quality 450mm quad frame. Glass fiber makes up the primary structure, whereas polyamide nylon makes up the arms. This version of the F450 has integrated PCB connections for soldering your ESCs directly to the board. This eliminates the need for a power distribution board or clumsy multi-connectors, resulting in a far cleaner electronic layout. On a hard landing, the F450 comes with stronger molded arms, which means no more arm breakage at the motor mount.



Figure 7:- F450 frame

Motor

Specifications:

- KV (rpm/volt): 1000
- Thrust: 900/4
- Efficiency: 80%
- efficiency current: 4-10A (>75%)
- Current capacity: 12A/60s
- Load current @10v: 0.5A
- Working voltage: 7-12V

Motor calculations:

To take off the UAV, the thrust should overcome the weight. when thrust is equal to weight, the UAV just hover and land. When,

$$\text{Thrust} = 2 * \text{mass} = 2 * 1$$

$$= 2 \text{ kg or } 2000 \text{ g.}$$

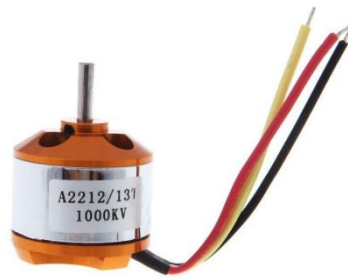


Figure 8:- Brushless motor

ESC (Electronic Speed Controller)

The ESC receives the Pulse Width Modulation (PWM) signal from the KK board and uses it to control the motors' speed. By obtaining electricity from a battery, the input signal is changed. It has a voltage reducer sensor that feeds the FS-i6 receiver and KK board with the needed output voltage battery.



Figure 9:- ESC

Propellers

The thrust required for the quadcopter to lift is generated by the propellers/fans. They are categorised and represented in the form of a product of diameter and pitch based on their diameter and pitch. The diameter of a propeller refers to the virtual circle it creates, whereas the pitch refers to the distance travelled per single rotation of the propeller. Quadcopters require two clockwise (CW) and two anticlockwise (CCW) rotating propellers to counteract motor torque. There are a variety of motors available with propeller specs for maximum consumption and power. If no propeller specifications can be found, the trial-and-error method must be utilized. As required, 10x4.5 black CW, CCW propellers in four sections are employed.



Figure 10:- Propellers

Lithium Ion Polymer (LiPo) Battery

Quadcopters are powered by Lithium Ion Polymer (LiPO) batteries. The batteries' cells have a voltage of 3.7 volts per cell and can generate a huge quantity of current for the BLDC motors. The C-ratings of these batteries are used to rank them. For a full charge, it takes two hours. According to Fig.8, a completely charged battery in a quadcopter can be utilised for 20 minutes depending on the payload. As a result, it can only offer 75A of source power. LIPO batteries must be used at least 80% of their full capacity.



Figure 11:- Battery

FLYSKY FS-i6 transmitter with receiver

The Fly Sky FS-i6 is a low-cost 6-channel 2.4 GHz transmitter and receiver that employs strong and dependable Automatic Frequency Hopping Digital System (AFHDS) spread spectrum technology.

The Fly Sky FS-i6 features a good quality look and feel, as well as straightforward programming. It also has a 6-channel receiver, the FS-iA6.



Figure 12:- Transmitter

KK2.1.5 flight controller

The KK2.1.5 Multi-Rotor Controller is a multi-rotor aircraft flight control board (Tricopters, Quadcopters, Hex copters etc.). Its job is to keep the plane stable during flight. It accomplishes this by receiving the signal from the 6050MPU gyro/acc (roll, pitch, and yaw) and sending it to the Atmega644PA IC.

The Atmega644PA IC unit subsequently processes these signals according to the firmware selected by the user and sends control signals to the Electronic Speed Controllers that have been installed (ESCs). These signals tell the ESCs to fine-tune the rotational speed of the motors, which stabilises your multi-rotor craft.

The Atmega644PA IC is controlled by the aileron, elevator, throttle, and rudder inputs on the KK2.1.5 Multi-Rotor control board, which additionally uses signals from your radio system's receiver (Rx). After processing this data, the IC sends different signals to the ESCs, which modify the rotational speed of each motor to achieve controlled flight.

A 6 pin AVR interface is built into the KK 2.1.5 flight controller board. It is powered by an Atmega644 PA processor chip and requires a 4.8V-6.0V supply voltage. The ESC motor receives a signal of 1520us from the KK 2.1. 5 flight controller.



Figure 13:- Flight controller

Arduino

The Arduino UNO is a microcontroller board that uses the ATmega328P microcontroller. It contains 14 digital input/output pins (including 6 PWM outputs), 6 analogue inputs, a 16 MHz ceramic resonator, a USB connection, a power jack, an ICSP header, and a reset button.

It comes with everything you'll need to get started with the microcontroller; simply plug it into a computer with a USB cable or power it with an AC-to-DC adapter or battery.



Figure 14:-Arduino UNO

Ultrasonic sensor



Figure 15:- ultrasonic sensor

An ultrasonic sensor is an electronic device that uses ultrasonic sound waves to detect the distance between a target item and converts the reflected sound into an electrical signal. Ultrasonic waves travel quicker than audible sound waves (i.e. the sound that humans can hear). The transmitter (which generates sound using piezoelectric crystals) and the receiver are the two primary components of ultrasonic sensors (which encounters the sound after it has travelled to and from the target).

The transmitter (which generates sound using piezoelectric crystals) and the receiver are the two primary components of ultrasonic sensors (which encounters the sound after it has travelled to and from the target).

$$D = \frac{1}{2} T \times C$$

(where D is the distance, T is the time, and C is the speed of sound ~ 343 meters/second).

Jumper wires



Figure 16:- jumper cables

Jumper wires are simply wires that have connector pins at each end, allowing them to be used to connect two points to each other without soldering. Jumper wires are typically used with bread board and other prototyping tools in order to make it easy to change a circuit as needed.

Methodology

Assembling the drone step by step

1. Assembling frames and setting up ESC's

To begin, connect all four ESCs to the bottom PCB (the one with the larger size will be the bottom plate), with the red wire soldered to the + sign point and the black wire soldered to the - terminal.

To solder the wire, first tin the spot on the board by putting solder to it, then solder out all of the ESC's clearly and firmly. Then, using the screws included with the frame, secure the four arms to the bottom plate.

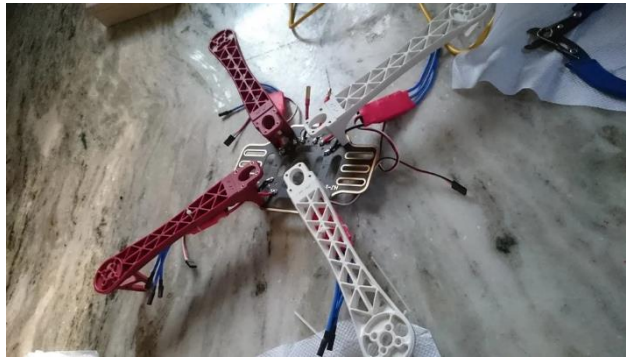


Figure 17:- Drone frame

2. Mounting the Motors

You can use zip ties or screws to mount the motors; we recommend screws but have used zip ties. After you've completed attaching the motors to the arms, connect the motor wires to the ESCs, zip tie all of the ESCs to the arms firmly, and secure the battery strap to the bottom board, then screw up the upper plate.



Figure 18:- Mounting motors

3. Installing KK2.1.5 Flight Controller and Receiver and mounting it on frame

Initial Setup of the KK2.1.5 disconnected from Multicopter

- Disconnect the Multicopter from the KK2.1.5.
- Use a four-cell battery pack to power the KK2.1.5.
- Choose the appropriate Motor Layout.
- In the Mode Settings, choose the Receiver Type.
- Turn off the KK2.1.5.
- Create a simple aircraft with no mixing in your transmitter.
- Bind your transmitter to the receiver if necessary.
- Switch on your transmitter.
- Connect the KK2.1.5 to the receiver.
- Turn on the KK2.1.5 and the receiver.
- Set up correctly according to the Receiver Test section of this manual.
- Turn off the KK2.1.X5 and the receiver.
- Turn off your transmitter.

Setup of the KK2.1.5 mounted in the Multicopter

- Take out all of the propellers.
- Place the KK2.1.5 on the frame such that the LCD is on the front and the buttons are on the rear.
- Connect the receiver to the left-hand pins. The negative (black or brown) points in the direction of the KK2.1.5's periphery
- Connect the ESCs and servos to the right-hand pins. Negative (brown or black) lead in the direction of the Kk2.1.5
- If necessary, connect a buzzer and a battery voltage monitor.
- Switch on your radio.
- First, the ESCs' throttle limits must be established.
- Set the maximum throttle on your transmitter.
- Press and hold the KK2.1.5's buttons 1 and 4 together.
- Activate the Multicopter's power.

Arm the KK2.1.X by lowering the throttle to the lowest setting and yawing to the right for a few seconds before applying a little amount of throttle. Ensure that all motors are spinning in the same direction as the Motor Layout. Remove two of the three motor wires from the ESC and change them over if a motor has to be reversed. It is not possible to utilise the KK2.1.X to reverse the direction of the motor.

If you put the roll/pitch stick towards one of the motors while they are going, it should slow down and the opposite motor should speed up. Check the stick directions in Receiver if this isn't the case.

- Dip one of the motors when the motors are operating and the sticks are in the centre to see if it will speed up. Check the stick directions in Receiver if this isn't the case.t.
- Move the throttle to minimum and yaw to the left for a few seconds to disarm the KK2.1.5.
- Turn off your multicopter
- Turn off your transmitter

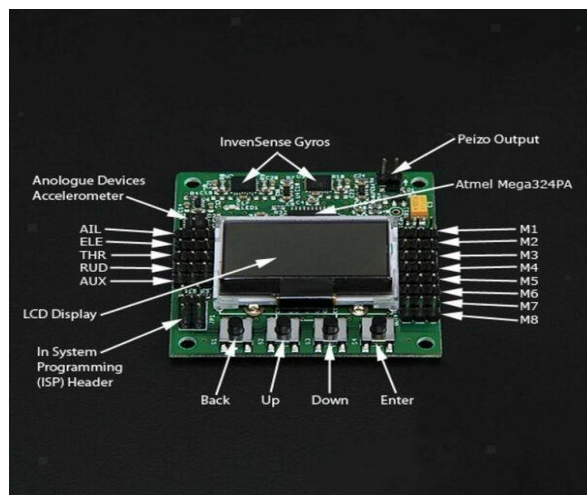


Figure 19:- Connection of KK 2.1.5 board

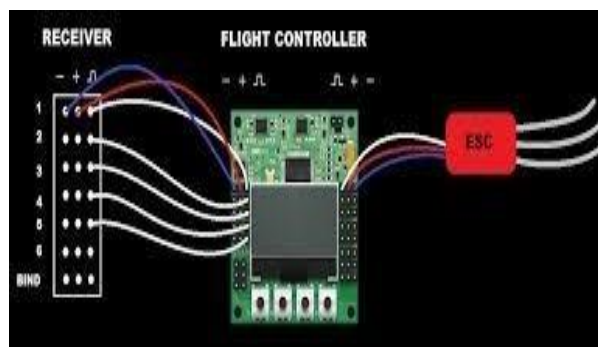


Figure 20:- Receiver and ESC connection



Figure 21:- KK 2.1.5 board mounted on to the drone

Using Velcro to secure the flying controller to the top plate's centre. The white arrow represents the front of the kk2 board, and the direction in which you fix it determines the front limbs or arms of your drone, so make the right decision before installing it.

Connect the ESC's signal wire to the board's right side pins; your first motor pins must be linked to the first row on the controller, and so on; and the black wire should run out of the board,

indicating that the rightmost pins on the flight controller are -ve terminals. Install the receiver on one of the arms and connect the wires to the receiver and the kk2's left side pins. On your receiver, channel 3 is usually throttle, channel 1 is elevator, channel 2 is roll (pitch), channel 4 is rudder, and channel 5 is usually Auxiliary (aux), but it can also be used for camera controls. The same things happen in the kk 2 board. The first row pins on the left side will be for the elevator, which controls the backward and forward motions of your quad; channel 2 means the second row of pins will be for roll pitch, which controls the side movements or rolling actions of your drone; channel 3 will be the throttle, which controls the motors speed; and finally, channel 4 will be for rotation of your quad at a fixed position; and aux is optional, which will be on channel 5, and you can also use it to switch on and off for segways.

4. Integrating Arduino and ultra-sonic sensor

Steps

1. First, connect the wires as shown in the diagram.
2. Open the Arduino IDE Software and create your code, or download and open the code below.
3. Select Tools > Board > Arduino/Geniuno Uno to select your own Arduino board (in this case, Arduino Uno).
4. Tools > Port > COM, then select your COM Port (typically there is only one). (If there are multiple ports, try each one separately.)
5. Press Ctrl + U or Sketch > Upload to upload your code.

You can use Serial Monitor to display the measurement data by pressing Ctrl + Shift+ M. (make sure that the baudrate speed is 9600).

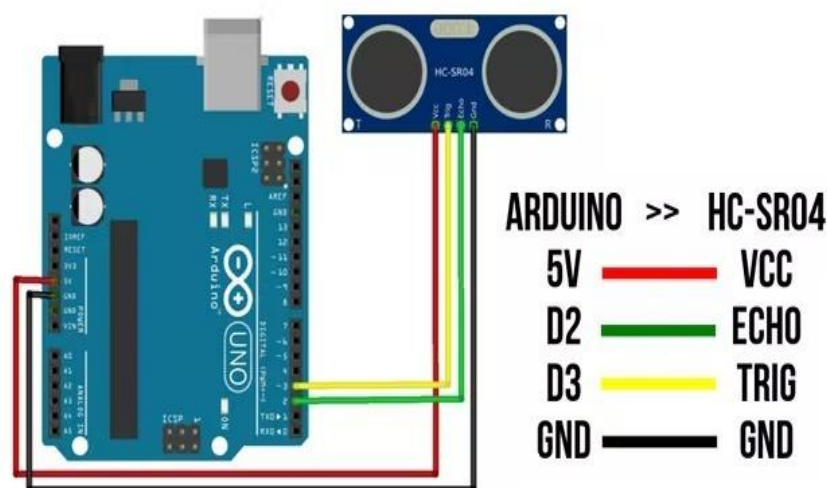


Figure 22:- connection between Arduino and sensor

Steps in Arduino software

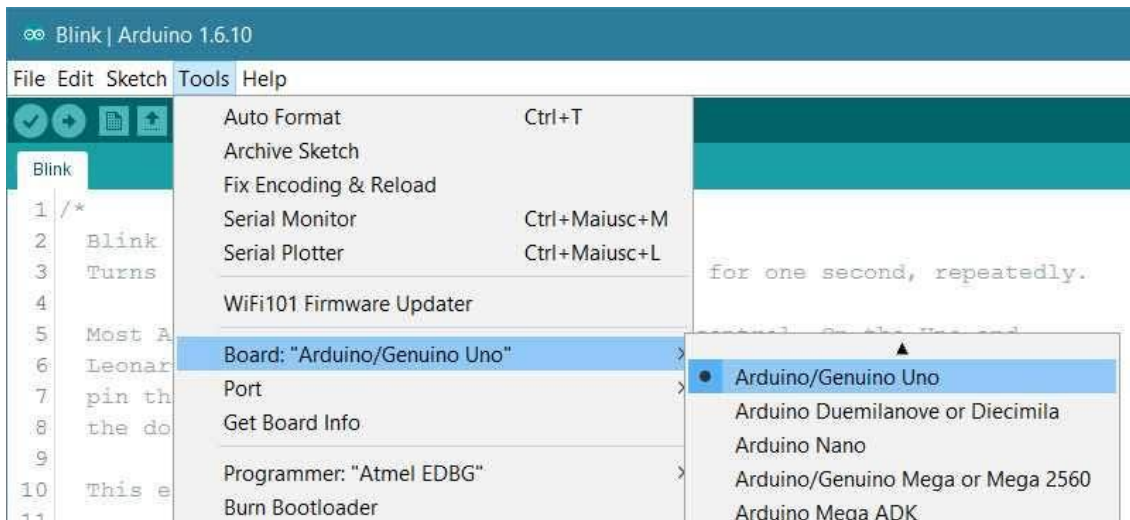


Figure 23:-Setup in arduino software

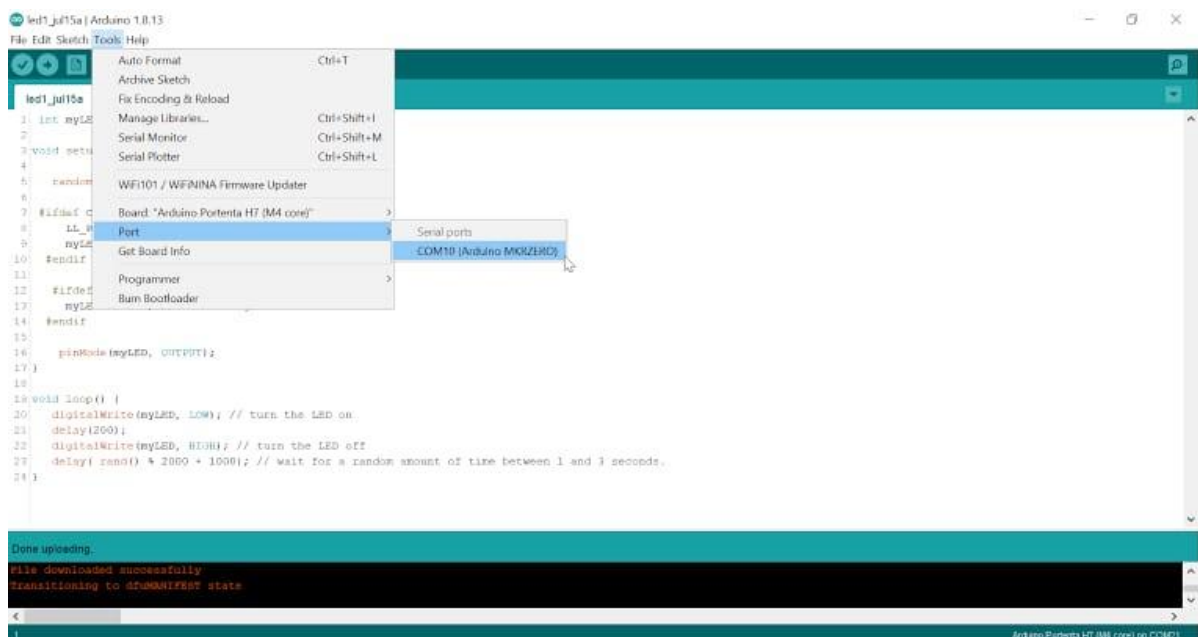


Figure 24:- Setup in Arduino software

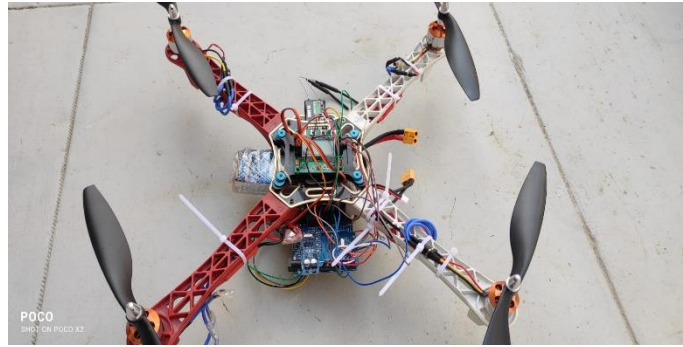


Figure 25:- Assembled drone



Figure 26:- Assembled drone

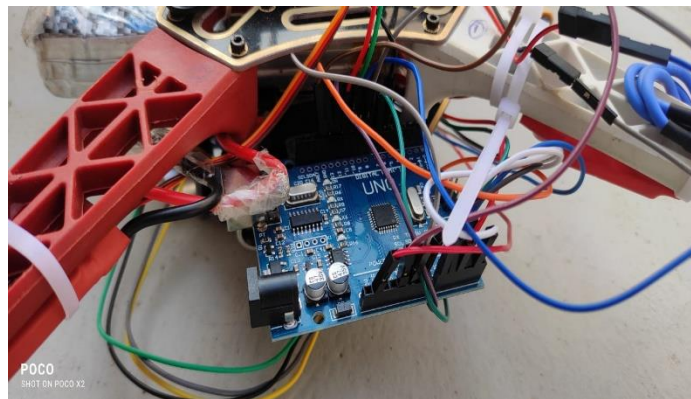


Fig 27:- Assembled drone

CHAPTER 2

LITERATURE SURVEY

A Survey on Unmanned Aerial Vehicle Collision Avoidance Systems.

Collision avoidance is a key factor in enabling the integration of unmanned aerial vehicle into real life use, whether it is in military or civil application. For a long time, there have been a large number of works to address this problem therefore, a comparative summary of them would be desirable. This paper presents a survey on the major collision avoidance systems developed in up to date publications. Each collision avoidance system contains two main parts sensing and detection, and collision avoidance. Based on their characteristics each part is divided into different categories; and those categories are explained, compared and discussed.

Unmanned Aerial Vehicles Collision Avoidance Systems and Approaches.

Moving towards autonomy, unmanned vehicles rely heavily on state-of-the-art collision avoidance systems (CAS). A lot of work is being done to make the CAS as safe and reliable as possible, necessitating a comparative study of the recent work in this important area. The paper provides a comprehensive review of collision avoidance strategies used for unmanned vehicles, with the main emphasis on unmanned aerial vehicles (UAV). It is an in-depth survey of different collision avoidance techniques that are categorically explained along with a comparative analysis of the considered approaches w.r.t. different scenarios and technical aspects. This also includes a discussion on the use of different types of sensors for collision avoidance in the context of UAVs.

Dynamic Target Tracking and Obstacle Avoidance using a Drone

This paper focuses on tracking dynamic targets using a low cost, commercially available drone. The approach presented utilizes a computationally simple potential field controller expanded to operate not only on relative positions, but also relative velocities. A brief background on potential field methods is given, and the design and implementation of the proposed controller is presented. Experimental results using an external motion capture system for localization demonstrate the ability of the drone to track a dynamic target in real time as well as avoid obstacles in its way

Vision Based Drone Obstacle Avoidance

Research on autonomous obstacle avoidance of drones has recently received widespread attention from researchers. Among them, an increasing number of researchers are using machine learning to train drones. These studies typically adopt supervised learning or reinforcement learning to train the networks. Supervised learning has a disadvantage in that it takes a significant amount of time to build the datasets, because it is difficult to cover the complex and changeable drone flight environment in a single dataset.

Reinforcement learning can overcome this problem by using drones to learn data in the environment. However, the current research results based on reinforcement learning are mainly focused on discrete action spaces. In this way, the movement of drones lacks precision and has somewhat unnatural flying behavior.

This study aims to use the soft-actor-critic algorithm to train a drone to perform autonomous obstacle avoidance in continuous action space using only the image data. The algorithm is trained and tested in a simulation environment built by Airsim. The results show that our

algorithm enables the UAV to avoid obstacles in the training environment only by inputting the depth map. Moreover, it also has a higher obstacle avoidance rate in the reconfigured environment without retraining.

Efficient Optical Flow and Stereo Vision for Velocity Estimation and Obstacle Avoidance on an Autonomous Pocket Drone

Micro Aerial Vehicles (FOV) are very suitable for flying in indoor environments, but autonomous navigation is challenging due to their strict hardware limitations. This paper presents a highly efficient computer vision algorithm called Edge-FS for the determination of velocity and depth. It runs at 20 Hz on a 4 g stereo camera with an embedded STM32F4 microprocessor (168 MHz, 192 kB) and uses edge distributions to calculate optical flow and stereo disparity. The stereo-based distance estimates are used to scale the optical flow in order to retrieve the drone's velocity. The velocity and depth measurements are used for fully autonomous flight of a 40 g pocket drone only relying on on-board sensors. This method allows the MAV to control its velocity and avoid obstacles.

Development of Autonomous Drones for Adaptive Obstacle Avoidance in Real World Environments

Recently, drones have been involved in several critical tasks such as infrastructure inspection, crisis response, and search and rescue operations. Such drones mostly use sophisticated computer vision techniques to effectively avoid obstacles and, thereby, require high computational power. Therefore, this work tuned and tested a computationally inexpensive algorithm, previously developed by the authors, for adaptive obstacle avoidance control of a drone. The algorithm aims at protecting the drone from entering in complex situations such as deadlocks and corners. The algorithm has been validated through simulation and implemented on a newly developed drone platform for infrastructure inspection. The design of the drone platform and the experimental results are presented in this study.

Obstacle Avoidance Drone by Deep Reinforcement Learning and Its Racing with Human Pilot

Drones with obstacle avoidance capabilities have attracted much attention from researchers recently. They typically adopt either supervised learning or reinforcement learning (RL) for training their networks. The drawback of supervised learning is that labeling of the massive dataset is laborious and time-consuming, whereas RL aims to overcome such a problem by letting an agent learn with the data from its environment.

The present study aims to utilize diverse RL within two categories discrete action space and discontinuous action space. The former has the advantage in optimization for vision datasets, but such actions can lead to unnatural behavior. For the latter, we propose a U-net based segmentation model with an actor-critic network.

Performance is compared between these RL algorithms with three different environments such as the woodland, block world, and the arena world, as well as racing with human pilots. Results suggest that our best continuous algorithm easily outperformed the discrete ones and yet was similar to an expert pilot.

Collision Avoidance System for Drones using “Ultrasonic sensor”

Collision Avoidance System for Drones, as the name suggests, protects the drone from collision from any obstacle such as a wall. In day to day life we hear drone crashes which cause machine loss as well as a great threat to people around. As drones are becoming common these days so it increases the threat of drone failures.

The idea behind the project is too simple. Whenever any obstacle is detected from left side the drone must drift towards right and same in all four directions. By keeping everything really simple we just manipulated the receiver's signals which are going to the flight controller chip to detect the obstacle, we used "Ultrasonic Sensors".

CHAPTER 3

ALGORITHM AND IMPLEMENTATION

Hardware design

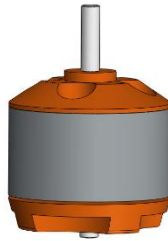


Figure 28:- 1000kv Brushless motor design

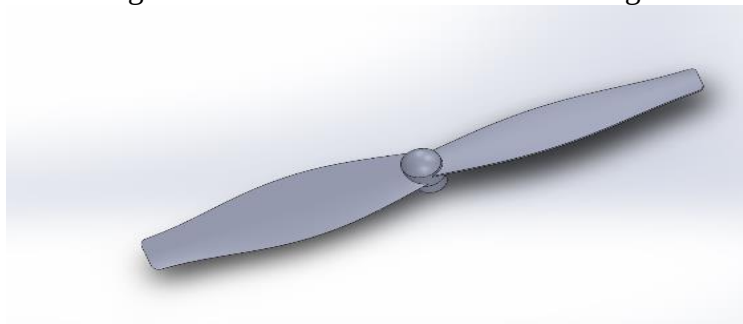


Figure 29:- Propeller design

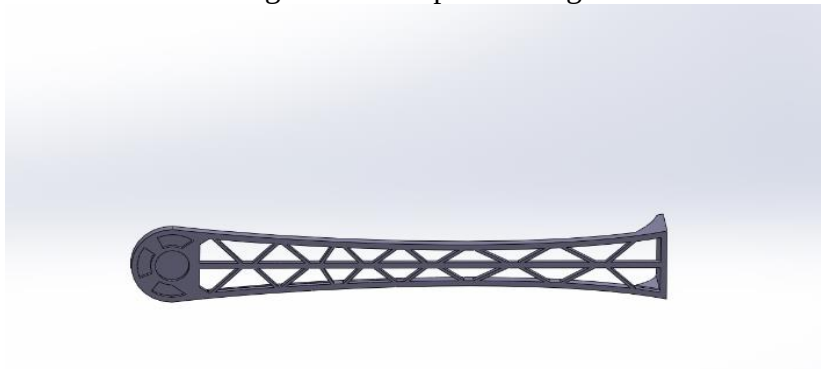


Figure 30:- Drone arm

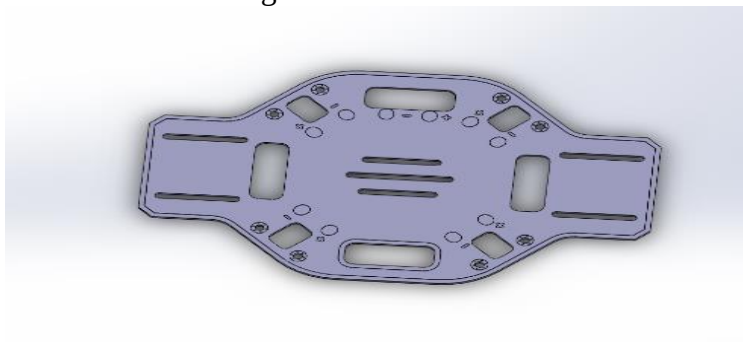


Figure 31:- F450 lower plate

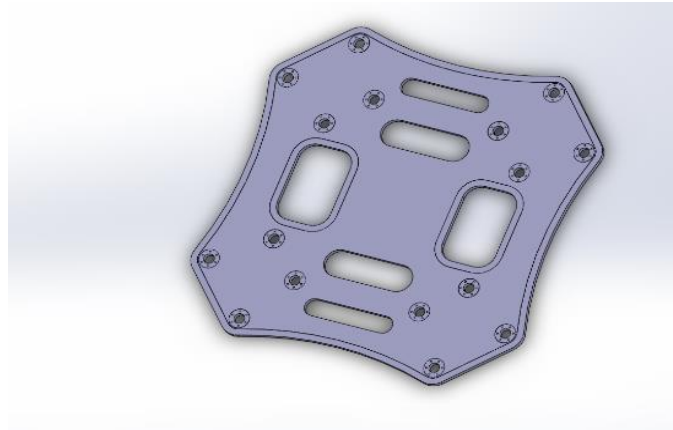


Figure 32:- F450 upper plate

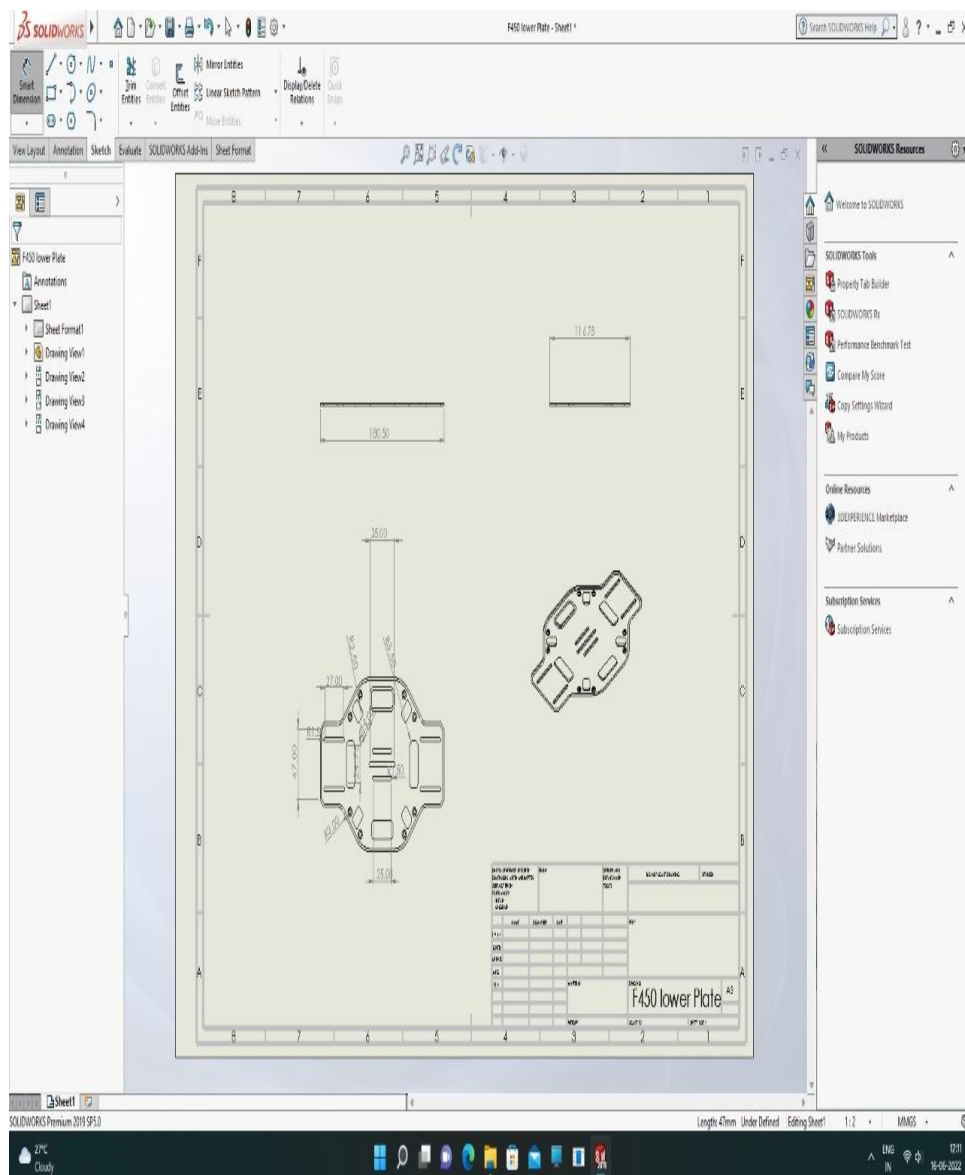


Figure 33:- F450 lower plate dimensions

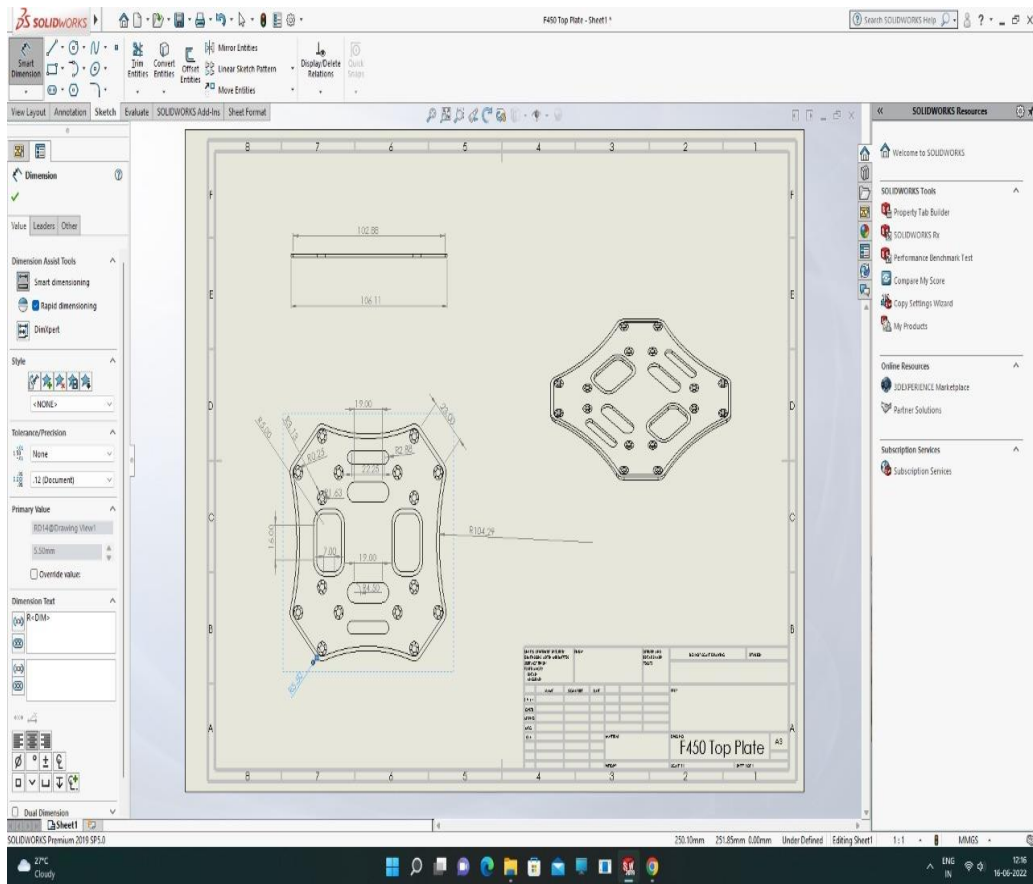


Figure 34:- F450 upper plate dimensions

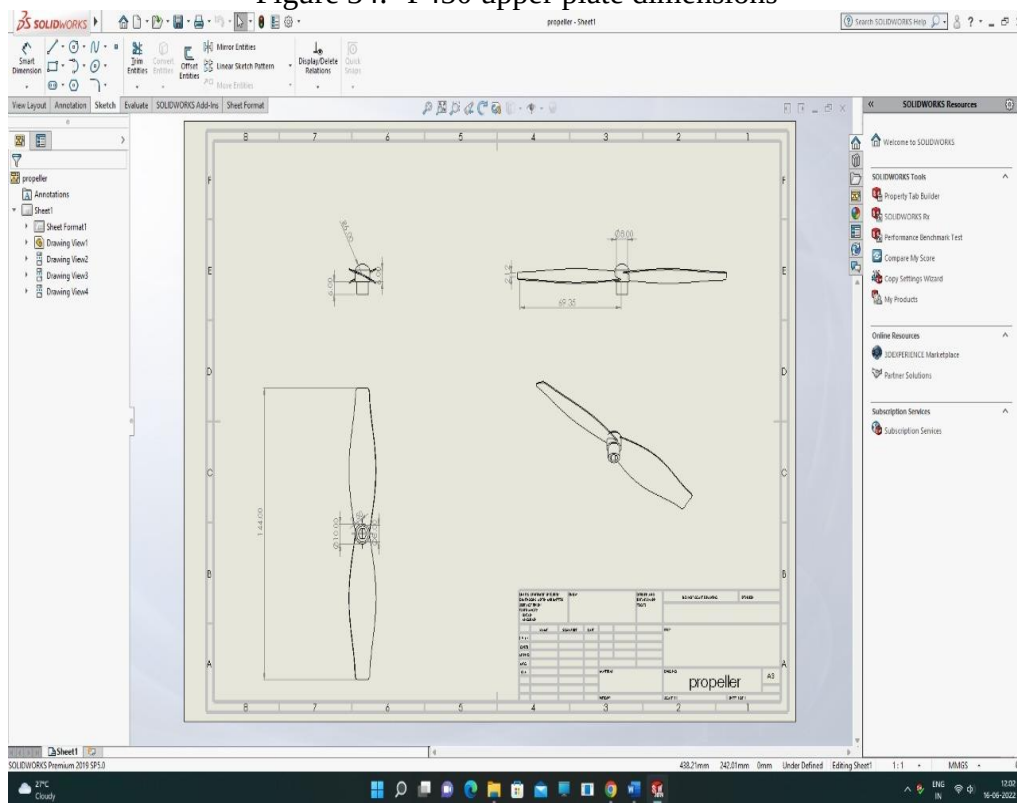


Figure 35:- F450 propeller dimension

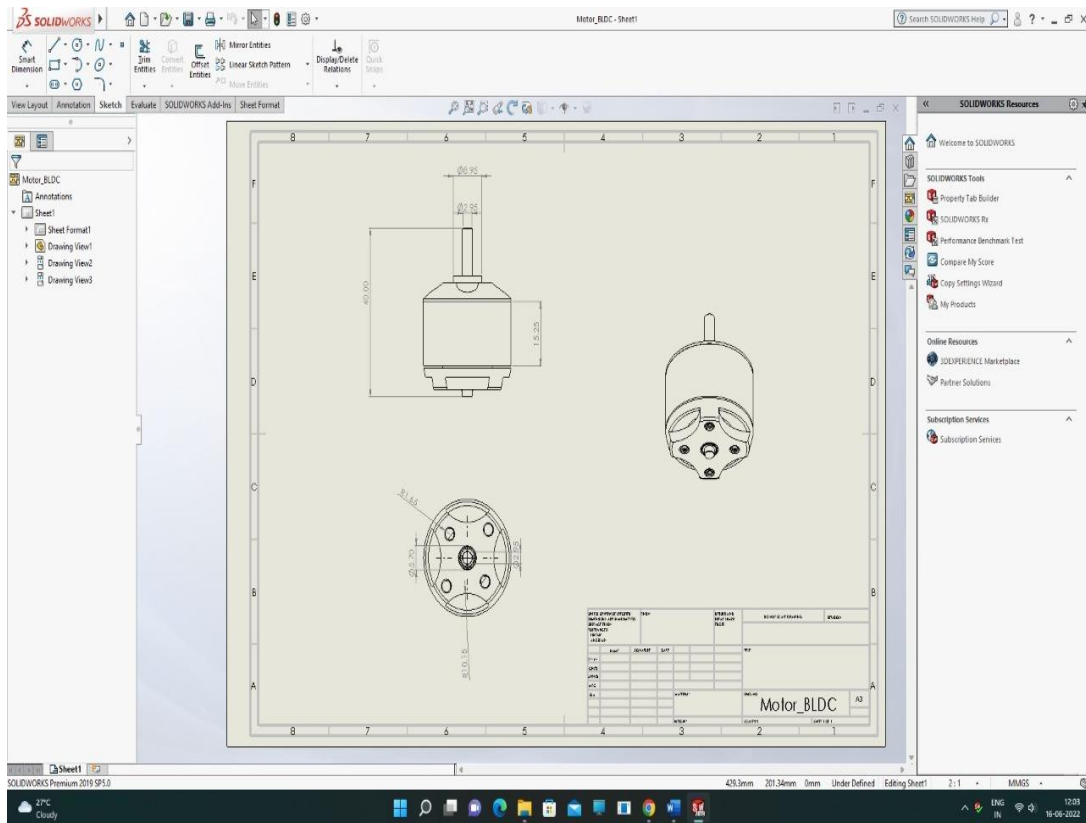


Figure 36:- Motor dimensions

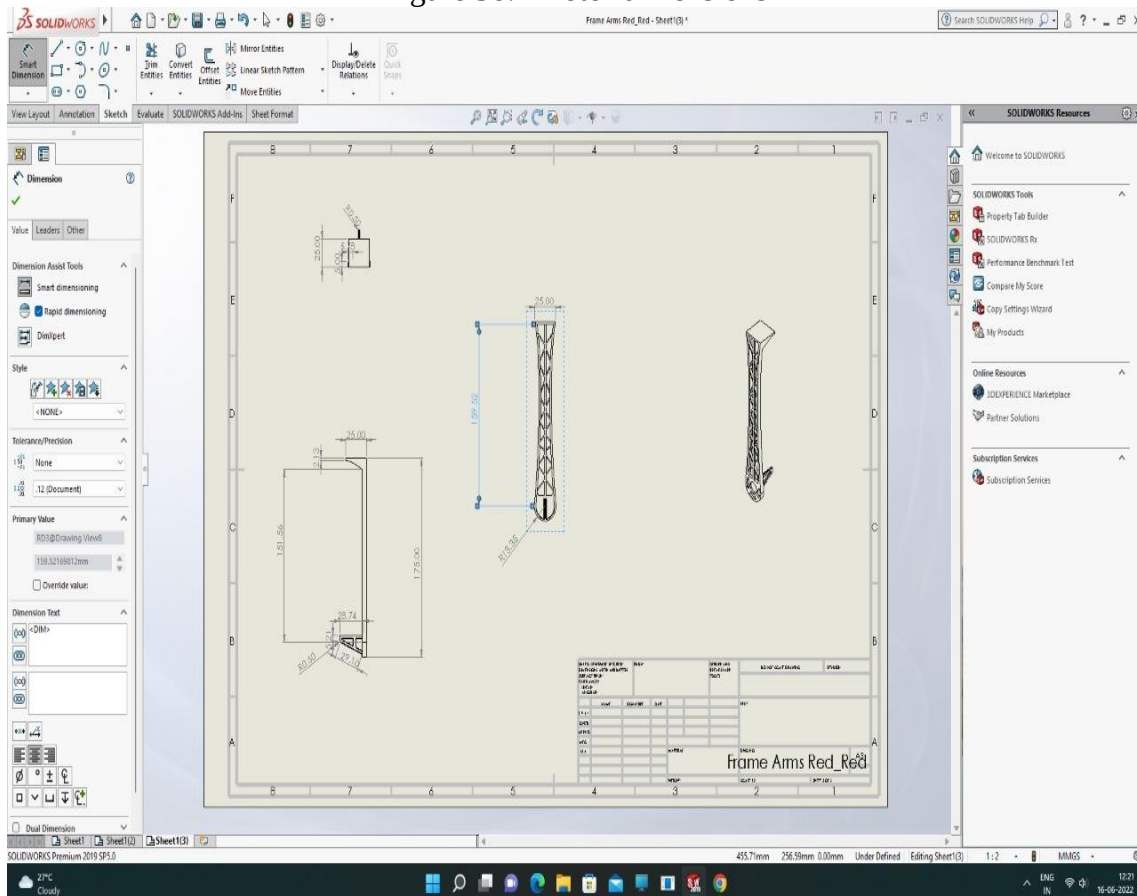
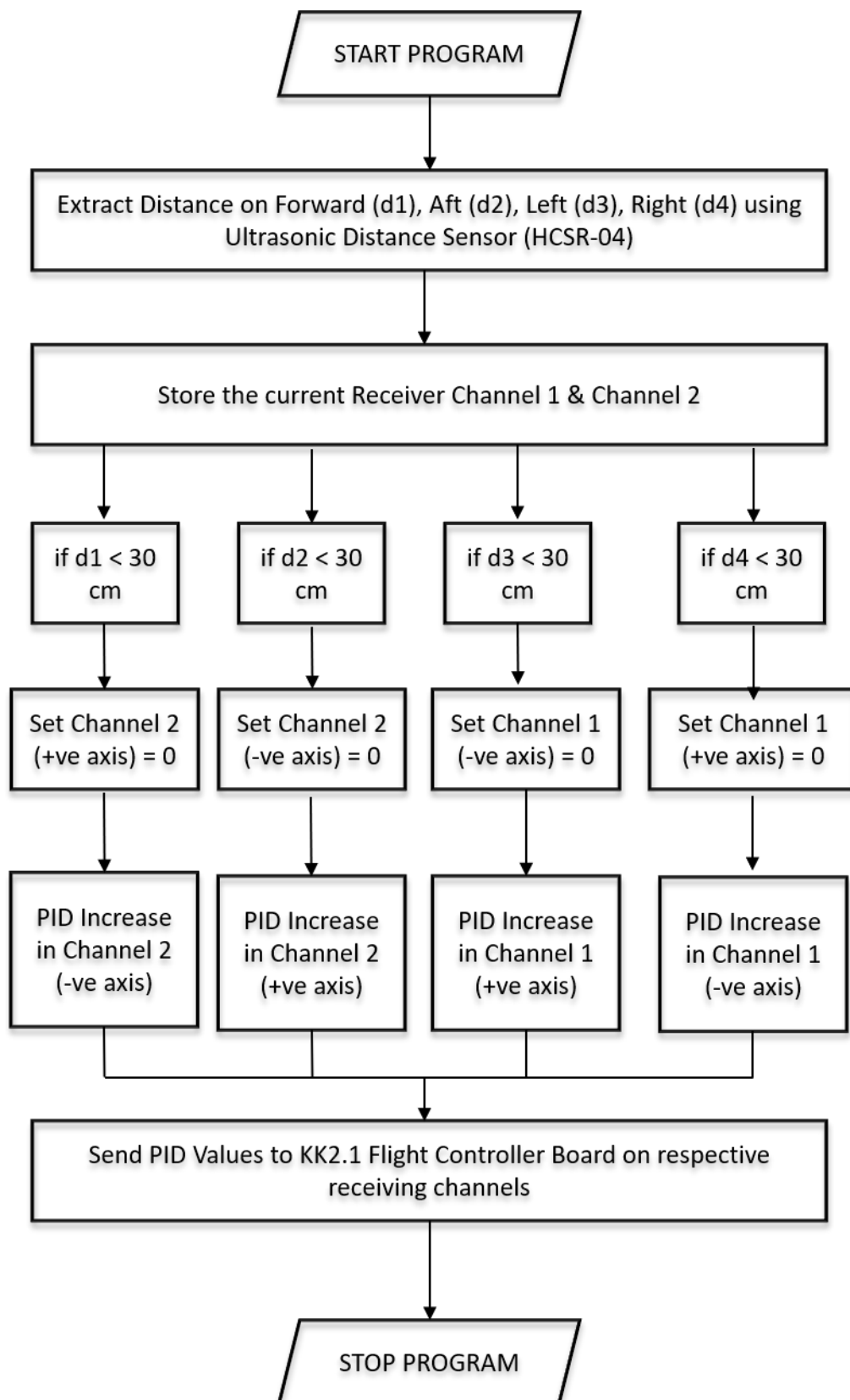


Figure 37:-Arm dimension

Flow chart for drone obstacle avoidance



We used a simple algorithm that is shown in the above flow chart.

When we start the program the first thing we should know is the distance of the object.

So what we have done is get the distance from the 4 sensors namely Forward(d1), Aft (2), Left(d3), Right(d4).

Once the sensors measure the distance, those values are stored into the receiver channels 1&2. If each sensors measures the distance less than 50 (since ultrasonic range is 200cm or 30kHz) each of those values are set to 2 different channels namely channel 1 (+ve &-ve) channel 2 (+ve &-ve).

Set channel 2 (+ve) is Forward

Set channel 2 (-ve) is Backward

Set channel 1 (+ve) is Left

Set channel 1 (-ve) is Right

These are then sent to PID which on increase will send it to FC (Flight Controller) through respective channels and stop the program.

PID (Proportional integral derivative) is the part of flight controller software that reads the data from the sensors and calculates how fast the motors should spin in order to retain desired rotation speed of UAV.

Raw code for drone obstacle avoidance

```
#define SONAR_NUM 4 -----define total number of Ultrasonic Sensors, in our case
we are using 4 sensors
#define MAX_DISTANCE 200 -----define the maximum distance, this is an optional
definition because our project uses the minimum distance rather than maximum distance
#include <NewPing.h> -----Install the New Ping Library which is basically a file
which extracts distances from unlimited connected Ultrasonic Distance Sensors with Arduino
UNO
```

```
int d1, d2, d3, d4, ch1, ch2; -----define 4 variables which store distance from 4 Ultrasonic
Sensors & ch1 & ch2 stores data from Transmitter Channel 1 & 2 (Roll & Pitch)
```

```
NewPing sonar[SONAR_NUM] = -----defining TRIG PIN & ECHO PIN connections
{
  NewPing (4, 5, MAX_DISTANCE), -----one Ultrasonic sensor's TRIG & ECHO PINS
are connected to Arduino Pin 4 & 5 respectively
  NewPing (6, 7, MAX_DISTANCE), -----one Ultrasonic sensor's TRIG & ECHO PINS
are connected to Arduino Pin 6 & 7 respectively
  NewPing (8, 9, MAX_DISTANCE), -----one Ultrasonic sensor's TRIG & ECHO PINS
are connected to Arduino Pin 8 & 9 respectively
  NewPing (10, 11, MAX_DISTANCE) -----one Ultrasonic sensor's TRIG & ECHO
PINS are connected to Arduino Pin 10 & 11 respectively
};
void setup () -----this portion defines what Pins on Arduino will be used as input &
which as output
{
  Serial.begin(115200); ----- at what rate should data be read & processed (bytes/second)
[115200 is called baud rate]
```

```

pinMode (2, INPUT); ----- Arduino Pin 2 will be used as input [Data will be taken
from Channel 1 of receiver]
pinMode (3, INPUT); ----- Arduino Pin 3 will be used as input [Data will be taken
from Channel 2 of receiver]
pinMode (1, OUTPUT); ----- Arduino Pin 1 will be used as output [Data will be
sent to Channel 1 of KK Board]
pinMode (12, OUTPUT); ----- Arduino Pin 12 will be used as output
[Data will be sent to Channel 2 of KK Board]
}

```

```

void loop ()
{
  d1 = sonar [0]. ping_cm (); -----calculate distance from 1st sensor and store to
variable d1
  d2 = sonar [1]. ping_cm (); -----calculate distance from 2nd sensor and store
to variable d2
  d3 = sonar [2]. ping_cm (); -----calculate distance from 3rd sensor and store
to variable d3
  d4 = sonar [3]. ping_cm (); -----calculate distance from 4th sensor and store
to variable d4

```

```

  ch1 = pulseIn (2, HIGH, 25000); -----start taking data from Receiver channel 1
(Arduino Pin 2) & store to variable ch1
  ch2 = pulseIn (3, HIGH, 25000); -----start taking data from Receiver channel 2
(Arduino Pin 3) & store to variable ch2

```

```

  mod_ch1 = map (ch1, 1000, 2000, -500, 500); -----adjust whatever value is
received from channel 1 of receiver between -500 & 500
  mod_ch2 = map (ch2, 1000, 2000, -500, 500); -----adjust whatever value is
received from channel 2 of receiver between -500 & 500

```

```

  if(d1<30) -----if distance on first sensor is less than 30 cm
  {
    mod_ch2 = (d1/30) *(-500); -----take the drone away from obstacle
automatically by making the drone go in opposite direction
    analog Write (1, mod_ch2);
  }

```

```

  if(d2<30)
  {
    mod_ch2 = (d2/30) *(500); -----take the drone away from obstacle
automatically by making the drone go in opposite direction
    analog Write (1, mod_ch2);
  }

```

```

  if(d3<30)
  {
    mod_ch1 = (d3/30) *(500); -----take the drone away from obstacle
automatically by making the drone go in opposite direction
    analog Write (1, mod_ch2);
  }

```

```

}

if(d4<30)
{
  mod_ch2 = (d4/30) *(-500); -----take the drone away from obstacle
  automatically by making the drone go in opposite direction
  analog Write (1, mod_ch2);
}
}

```

Implemented code

```

#include <NewPing.h>
#include <Servo.h>

#define SONAR_NUM 4
#define MAX_DISTANCE 200

//connect 4 HCSR04 on (4,5)[left], (6,7)[right], (8,9)[forward], (10, 11)[backward];

NewPing sonar[SONAR_NUM] =
{
  NewPing(12, 7, MAX_DISTANCE),
  NewPing(16, 17, MAX_DISTANCE),
  NewPing(13, 18, MAX_DISTANCE),
  NewPing(14, 15, MAX_DISTANCE)
};

int trigger;
int roll;
int pitch;
int roll_to_kk;
int pitch_to_kk;
int obstacle_left;
int obstacle_right;
int obstacle_forward;
int obstacle_backward;
int a =1;
int b=1;
Servo out1; //roll
Servo out2; //pitch

int sensitivity = 4; //Drone Response Sensitivity can be changed from here, large number will
result in lower sensitivity & vice versa

void setup()
{
  Serial.begin(115200);
  pinMode(10, INPUT); //connect Channel 5 from reciever to arduino Pin 14 (analog A0)
  pinMode(5, INPUT); //connect roll channel from reciever to Arduino Pin 2

```

```

pinMode(6, INPUT); //connect pitch channel from reciever to Arduino Pin 3
pinMode(8, OUTPUT); //connect Arduino Pin 12 to roll channel of KK Board
pinMode(9, OUTPUT); //connect Arduino Pin 13 to pitch channel of KK Board
out1.attach(9); //pitch
out2.attach(8); //roll
}

void loop()
{
trigger = pulseIn(10, HIGH);
roll = pulseIn(11, HIGH);
pitch = pulseIn(6, HIGH);
obstacle_right = int(sonar[0].ping_cm());
obstacle_left = int(sonar[1].ping_cm());
obstacle_backward = int(sonar[2].ping_cm());
obstacle_forward = int(sonar[3].ping_cm());

//roll = map(roll,990, 2000, 0, 500);
//pitch = map(pitch, 990, 2000, 0, 500);
Serial.print("T: ");
Serial.print(trigger);
Serial.print(" ");
Serial.print("R: ");
Serial.print(roll);
Serial.print(" ");
Serial.print("P: ");
Serial.println(pitch);

/*
Serial.print("L: ");
Serial.print(obstacle_left);
Serial.print(" ");
Serial.print("R: ");
Serial.print(obstacle_right);
Serial.print(" ");
Serial.print("F: ");
Serial.print(obstacle_forward);
Serial.print(" ");
Serial.print("B: ");
Serial.println(obstacle_backward);
*/
if (trigger>1500)
{
b=1;
if(a==1)
{
Serial.println("-----Entered Obstacle Avoidance Mode-----");
delay(1000);
a=a+1;
}
}
}

```



```

}
if(obstacle_left < 50 && obstacle_left>5)
{
roll_to_kk = roll + int(1000/obstacle_left);
out2.write(roll_to_kk);
out1.write(pitch);
Serial.print(">> LEFT >>");
Serial.print(roll);
Serial.print("|");
Serial.print(roll_to_kk);
Serial.print("| OL: ");
Serial.println(obstacle_left);
//Serial.println("Obstacle on Left");
}

if(obstacle_right < 50 && obstacle_right>5)
{
roll_to_kk = roll - int(1000/obstacle_right);
out1.write(roll_to_kk);
out2.write(pitch);
Serial.print(">> RIGHT >>");
Serial.print(roll);
Serial.print("|");
Serial.print(roll_to_kk);
Serial.print("| OR: ");
Serial.println(obstacle_right);
//Serial.print("Obstacle on Right");
}

if(obstacle_forward < 50 && obstacle_forward>5)
{
pitch_to_kk = pitch - int(1000/obstacle_forward);
out2.write(pitch_to_kk);
out1.write(roll);
Serial.print(">> FORWARD >>");
Serial.print(pitch);
Serial.print("|");
Serial.print(pitch_to_kk);
Serial.print("| OF: ");
Serial.println(obstacle_forward);
//Serial.print("Obstacle Forward");
}

if(obstacle_backward < 50 && obstacle_backward>5)
{
pitch_to_kk = pitch + int(1000/obstacle_backward);
out2.write(pitch_to_kk);
out1.write(roll);
Serial.print(">> BACKWARD >>");
Serial.print(pitch);

```

```

Serial.print("|");
Serial.print(pitch_to_kk);
Serial.print("| OB: ");
Serial.println(obstacle_backward);
//Serial.print("Obstacle Backward");
}
/*
else
{
Serial.println("NO OBSTACLE");
out2.write(pitch);
out1.write(roll);
}
*/
}

else
{
if(b==1)
{
Serial.println("NORMAL FLYING MODE");
b=b+1;
delay(1000);
}
out1.write(roll);
out2 .write(pitch);
Serial.print("Roll: ");
Serial.print(roll);
Serial.print(" | ");
Serial.print("Pitch: ");
Serial.println(pitch);
a=1;
}

}

```



```

sketch_jun16a | Arduino 1.8.19 (Windows Store 1.8.57.0)
File Edit Sketch Tools Help

sketch_jun16a $

#include <NewPing.h>
#include <Servo.h>

#define SONAR_NUM 4
#define MAX_DISTANCE 200

//connect 4 HCSR04 on (4,5)[left], (6,7)[right], (8,9)[forward], (10, 11)[backward];

NewPing sonar[SONAR_NUM] =
{
  NewPing(12, 7, MAX_DISTANCE),
  NewPing(16, 17, MAX_DISTANCE),
  NewPing(13, 18, MAX_DISTANCE),
  NewPing(14, 15, MAX_DISTANCE)
};

int trigger;
int roll;
int pitch;
int roll_to_kk;
int pitch_to_kk;
int obstacle_left;
int obstacle_right;
int obstacle_forward;
int obstacle_backward;
int a =1;
int b=1;
Servo out1; //roll
Servo out2; //pitch

int sensitivity = 4; //Drone Response Sensitivity can be changed from here, large number will result in lower sensitivity & vice versa

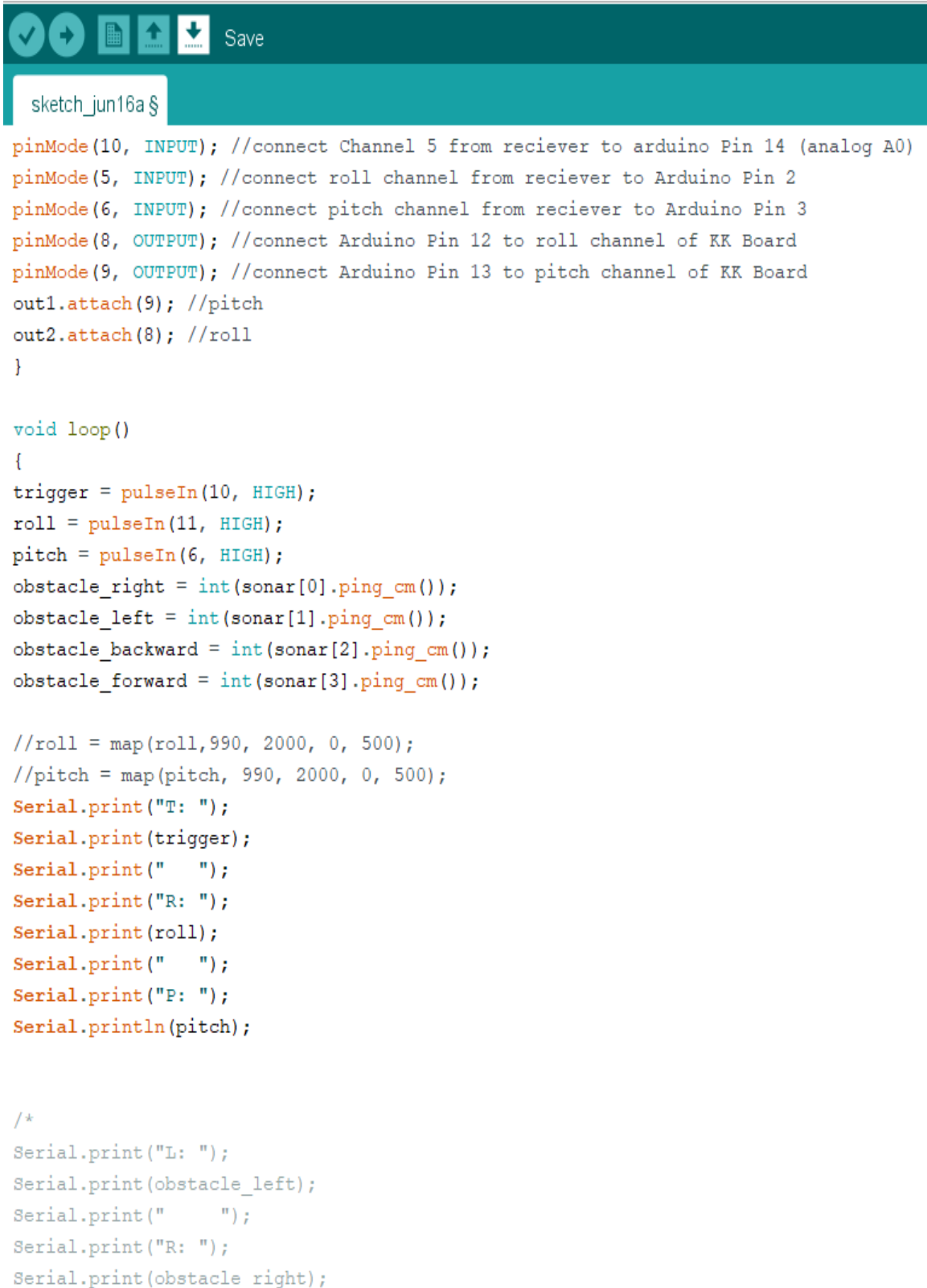
void setup()
{
  Serial.begin(115200);
  pinMode(10, INPUT); //connect Channel 5 from reciever to arduino Pin 14 (analog A0)
  pinMode(5, INPUT); //connect roll channel from reciever to Arduino Pin 2

```

Figure 38:- Obstacle avoidance code

sketch_jun16a | Arduino 1.8.19 (Windows Store 1.8.57.0)

File Edit Sketch Tools Help



```

pinMode(10, INPUT); //connect Channel 5 from reciever to arduino Pin 14 (analog A0)
pinMode(5, INPUT); //connect roll channel from reciever to Arduino Pin 2
pinMode(6, INPUT); //connect pitch channel from reciever to Arduino Pin 3
pinMode(8, OUTPUT); //connect Arduino Pin 12 to roll channel of KK Board
pinMode(9, OUTPUT); //connect Arduino Pin 13 to pitch channel of KK Board
out1.attach(9); //pitch
out2.attach(8); //roll
}

void loop()
{
trigger = pulseIn(10, HIGH);
roll = pulseIn(11, HIGH);
pitch = pulseIn(6, HIGH);
obstacle_right = int(sonar[0].ping_cm());
obstacle_left = int(sonar[1].ping_cm());
obstacle_backward = int(sonar[2].ping_cm());
obstacle_forward = int(sonar[3].ping_cm());

//roll = map(roll,990, 2000, 0, 500);
//pitch = map(pitch, 990, 2000, 0, 500);
Serial.print("T: ");
Serial.print(trigger);
Serial.print(" ");
Serial.print("R: ");
Serial.print(roll);
Serial.print(" ");
Serial.print("P: ");
Serial.println(pitch);

/*
Serial.print("L: ");
Serial.print(obstacle_left);
Serial.print(" ");
Serial.print("R: ");
Serial.print(obstacle_right);

```

Figure 39:- Obstacle avoidance code



The image shows the Arduino IDE interface with the sketch named 'sketch_jun16a'. The code is as follows:

```

Serial.print("R: ");
Serial.print(obstacle_right);
Serial.print(" ");
Serial.print("F: ");
Serial.print(obstacle_forward);
Serial.print(" ");
Serial.print("B: ");
Serial.println(obstacle_backward);
*/
if (trigger>1500)
{
b=1;
if(a==1)
{
Serial.println("-----Entered Obstacle Avoidance Mode-----");
delay(1000);
a=a+1;
}
if(obstacle_left < 50 && obstacle_left>5)
{
roll_to_kk = roll + int(1000/obstacle_left);
out2.write(roll_to_kk);
out1.write(pitch);
Serial.print(">> LEFT >>");
Serial.print(roll);
Serial.print("|");
Serial.print(roll_to_kk);
Serial.print("| OL: ");
Serial.println(obstacle_left);
//Serial.println("Obstacle on Left");
}

if(obstacle_right < 50 && obstacle_right>5)
{
roll_to_kk = roll - int(1000/obstacle_right);
out1.write(roll_to_kk);
out2.write(pitch);

```

Figure 40:- Obstacle avoidance code

sketch_jun16a | Arduino 1.8.19 (Windows Store 1.8.57.0)

File Edit Sketch Tools Help

sketch_jun16a \$

```

if(obstacle_forward < 50 && obstacle_forward>5)
{
pitch_to_kk = pitch - int(1000/obstacle_forward);
out2.write(pitch_to_kk);
out1.write(roll);
Serial.print(">> FORWARD >>");
Serial.print(pitch);
Serial.print("|");
Serial.print(pitch_to_kk);
Serial.print("| OF: ");
Serial.println(obstacle_forward);
//Serial.print("Obstacle Forward");
}

if(obstacle_backward < 50 && obstacle_backward>5)
{
pitch_to_kk = pitch + int(1000/obstacle_backward);
out2.write(pitch_to_kk);
out1.write(roll);
Serial.print(">> BACKWARD >>");
Serial.print(pitch);
Serial.print("|");
Serial.print(pitch_to_kk);
Serial.print("| OB: ");
Serial.println(obstacle_backward);
//Serial.print("Obstacle Backward");
}
/*
else
{
Serial.println("NO OBSTACLE");
out2.write(pitch);
out1.write(roll);
}
*/
}

```

Figure 41:- Obstacle avoidance code

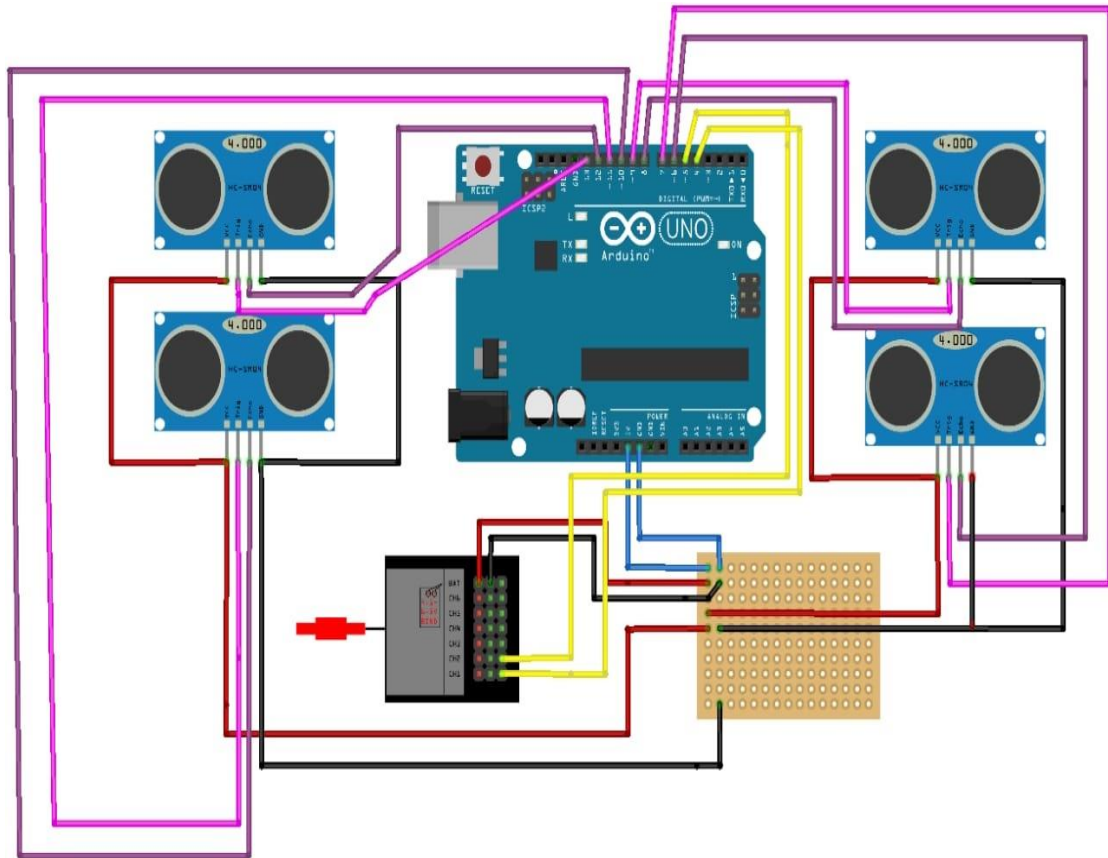


Figure 42:- Connections of 4 ultrasonic sensors to Arduino board

Hardware test

Hardware testing is of different types, functionality test and performance test.

Functionality test:- In functionality test our drone avoid Obstacles in range of 30 cm, and we kept separate mode for Obstacle Avoidance.

Performance test:- Overall payload Capacity of our drone is 1.2 (Kg) [Including the battery], and endurance is 9 mins.

Software validation

As we are using arduino software, so all validation that applies on Arduino Software Compatibility will be our validation too.

CHAPTER 4

RESULTS AND DISCUSSION

Discussion

The drone was able to meet the weight requirements. The total drone was approximately 1.8 kg and the power from the motors offered enough thrust for a 0.5 kg payload. The drone met the requirement to have four propellers and four motors.

During the first flight test propellers did not resist the damage, but the sensor module was protected and remained undamaged.

The drone conditionally met the requirement to have the ability to sense the object.

The drone met the requirement to be easy to carry and deploy, friendly, and intuitive. It was less than 4 cubic feet and was put together with well-placed screws and nuts

Usually for any drone to possess the capability to AVOID OBSTACLES, it must have LIDAR or any distance sensor which can be interfaced with the Flight Controller.

Our project demonstrates a low-cost but effective alternative to any expensive flight controller. This low-cost effective alternative is basically bypassing few channels of Controller to a low-cost Flight Controller via an Arduino Board.

This project basically uses 4 Ultrasonic Distance Sensors to detect obstacles in 4 directions, namely forward, backward, left & right.

These 4 Ultrasonic Sensors tell the Drone about its position with respect to any obstacle(s) around. The algorithm basically takes 2 channels from receiver (Pitch & Roll) and then based on obstacle present in any direction, Roll & Pitch Value is changed automatically so that Drone moves away from the obstacle.

Our Drone also responds with different sensitivities depending on how fast the drone is flying near any obstacle. This response rate can be changed by varying the Sensitivity factor given in the code.

RESULT:

1. Our project demonstrates a low-cost & effective alternative to use any LIDAR enabled Expensive Flight Controllers.
2. This project is a demonstration of Mandatory Safety Measure that should be present in Commercial Drones such as Wedding Photography Drone, Low Altitude Surveillance Drones, Agri-Drones etc, where both Human/Animal's safety and Drone's safety can be ensured using an Obstacle Avoiding feature.

COM3

```

Roll: 1487 | Pitch: 1495
T: 994   R: 1493   P: 1489
Roll: 1493 | Pitch: 1489
T: 994   R: 1493   P: 1488
Roll: 1493 | Pitch: 1488
T: 994   R: 1493   P: 1488
Roll: 1493 | Pitch: 1488
T: 994   R: 1493   P: 1489
Roll: 1493 | Pitch: 1489
T: 994   R: 1487   P: 1494
Roll: 1487 | Pitch: 1494
T: 994   R: 1493   P: 1488
Roll: 1493 | Pitch: 1488
T: 994   R: 1487   P: 1494
Roll: 1487 | Pitch: 1494
T: 1000   R: 1492   P: 1494
Roll: 1492 | Pitch: 1494
T: 994   R: 1493   P: 1489
Roll: 1493 | Pitch: 1489
T: 994   R: 1488   P: 1494
Roll: 1488 | Pitch: 1494
T: 994   R: 1494   P: 1494
Roll: 1494 | Pitch: 1494
T: 995   R: 1493   P: 1488
Roll: 1493 | Pitch: 1488
T: 1974   R: 1487   P: 1494
-----Entered Obstacle Avoidance Mode-----
T: 1989   R: 1493   P: 1489
>> FORWARD >>1489|1464| OF: 40
>> BACKWARD >>1489|1509| OB: 49
T: 1809   R: 1493   P: 1488
>> FORWARD >>1488|1463| OF: 39
>> BACKWARD >>1488|1508| OB: 49
T: 1809   R: 1493   P: 1488
>> FORWARD >>1488|1463| OF: 39
>> BACKWARD >>1488|1508| OB: 49
T: 1809   R: 1494   P: 1489
>> FORWARD >>1489|1464| OF: 39
>> BACKWARD >>1489|1509| OB: 49
T: 1807   R: 1487   P: 1494
>> FORWARD >>1494|1469| OF: 39
>> BACKWARD >>1494|1514| OB: 48
T: 1809   R: 1487   P: 1495
>> FORWARD >>1495|1470| OF: 40
>> BACKWARD >>1495|1515| OB: 49
T: 1801   R: 1488   P: 1494
>> FORWARD >>1494|1469| OF: 39

```

☒ Autoscroll ☐ Show timestamp

Figure 43:- Output of roll pitch values

COM3

```

T: 1988      R: 1488      P: 1494
>> RIGHT >>1488|1468| OR: 48
>> FORWARD >>1494|1383| OF: 9
>> BACKWARD >>1494|1515| OB: 46
T: 1988      R: 1494      P: 1488
>> RIGHT >>1494|1474| OR: 48
>> FORWARD >>1488|1377| OF: 9
>> BACKWARD >>1488|1509| OB: 46
T: 1988      R: 1495      P: 1488
>> RIGHT >>1495|1475| OR: 48
>> FORWARD >>1488|1377| OF: 9
>> BACKWARD >>1488|1510| OB: 45
T: 1988      R: 1495      P: 1488
>> RIGHT >>1495|1474| OR: 47
>> FORWARD >>1488|1377| OF: 9
>> BACKWARD >>1488|1510| OB: 45
T: 1989      R: 1494      P: 1488
>> RIGHT >>1494|1473| OR: 47
>> FORWARD >>1488|1377| OF: 9
>> BACKWARD >>1488|1509| OB: 46
T: 1988      R: 1488      P: 1494
>> RIGHT >>1488|1467| OR: 47
>> FORWARD >>1494|1383| OF: 9
>> BACKWARD >>1494|1515| OB: 46
T: 1988      R: 1488      P: 1495
>> RIGHT >>1488|1467| OR: 47
>> FORWARD >>1495|1384| OF: 9
>> BACKWARD >>1495|1517| OB: 45
T: 1988      R: 1488      P: 1495
>> RIGHT >>1488|1467| OR: 46
>> FORWARD >>1495|1384| OF: 9
>> BACKWARD >>1495|1517| OB: 45
T: 1988      R: 1488      P: 1494
>> RIGHT >>1488|1468| OR: 48
>> FORWARD >>1494|1383| OF: 9
>> BACKWARD >>1494|1516| OB: 45
T: 1988      R: 1495      P: 1488
>> RIGHT >>1495|1474| OR: 47
>> FORWARD >>1488|1377| OF: 9
>> BACKWARD >>1488|1511| OB: 43
T: 1988      R: 1494      P: 1488
>> RIGHT >>1494|1473| OR: 47
>> FORWARD >>1488|1377| OF: 9
>> BACKWARD >>1488|1510| OB: 45
T: 1989      R: 1495      P: 1488
>> RIGHT >>1495|1474| OR: 47
>> FORWARD >>1488|1377| OF: 9

```

☒ Autoscroll ☐ Show timestamp

Figure 44:- Output of obstacle avoidance code

CHAPTER 5

APPLICATIONS

When autonomous drones can avoid both static and dynamic obstacles, the possibilities are endless, including package delivery, infrastructure and pipeline inspections, and search and rescue, to mention a few. And there are a plethora of applications that have yet to be explored.

- **Safety:** This is the most important advantage of obstacle avoidance. No one wants to think of their drone crashing. Everyone desires for their device to be returned to them safely, but this does not always happen. You might lose attention from time to time, leading the machine to collide with an object and fly backward or to the side.
- If your drone has obstacle avoidance, on the other hand, it will avoid colliding with the object in the first place. This will also ensure the safety of those who are in the vicinity of where you are flying your drone.
- **Confident flight:** There's a lot of anxiety that comes with flying a drone if you're not convinced you'll be able to do so securely. However, if you use a drone with obstacle avoidance, you can feel comfortable piloting it and enjoy the flight. Whether you're flying a drone for commercial purposes or pleasure, obstacle avoidance will make a big impact once you're up in the air.
- **Indoor flight:** A drone can fly considerably more effectively indoors with the object avoidance feature. Because of the technology, your drone will be able to travel through narrower spaces while avoiding obstructions.
- **Future drone parcel delivery:** Drones are the future of delivery, and they'll soon be delivering medications, parcels, and other items to your doorstep. Several obstacles must be overcome in order for this to happen, and obstacle avoidance is a big part of it. It will ensure that the drones are safe to use and that no individual or property is endangered.
- **Insurance costs:** Insuring a professional aerial filming or multispectral drone can be prohibitively expensive. A top-of-the-line multirotor capable of transporting pricey camera equipment could cost up to \$50,000. Insurance is required for these drones, and the prices of insurance are expensive. These insurance expenses can be reduced by using a drone with obstacle recognition and collision avoidance capabilities.

CHAPTER 6

CONCLUSION AND FUTURE SCOPE

Future scope

When combined with a GPS module, this technology could allow the quadcopter to go unambiguously between any two specified co-ordinates without colliding.

- In search and rescue operations, the collision avoidance module is a big help. Natural disasters such as earthquakes and floods render certain locations inaccessible to humans. Drones with autonomy could be employed in these situations. Drones used for reconnaissance are equipped with such components.
- The quadcopter can map its surroundings and build a 3D map using a wireless workstation and a sophisticated GUI.
- Drones have long been used in military purposes. This drone, on the other hand, may be equipped with remote sensing light weaponry capable of piercing army bunkers and tanks.
- Another amazing idea is the nano-copter, which is based on existing quadcopters but with a smaller size

Conclusion

The quadcopter stabilises itself using the array of sensors integrated on it, as per the design parameters. The system is up and running, capable of avoiding obstacles and performing autonomous operations such as collision avoidance and position hold and change.

Overall, an unmanned aerial vehicle was fully realized in this project that could perform the feature of obstacle avoidance.

REFERENCE

1. Albaker, B.M.; Rahim, N.A., "A survey of collision avoidance approaches for unmanned aerial vehicles," in Technical Postgraduates (TECHPOS), 2009 International Conference for , vol., no., pp.1-7, 14-15 Dec. 2009.
2. A. Mcfadyen and L. Mejias, "A survey of autonomous vision-based see and avoid for unmanned aircraft systems", Prog. Aerosp. Sci., vol. 80, pp. 1-17, Jan. 2016.
3. https://scholar.google.co.in/scholar?q=Dynamic+Target+Tracking+and+Obstacle+Avoidance+using+a+Drone&hl=en&as_sdt=0&as_vis=1&oi=scholar.
4. Dynamic Target Tracking and Obstacle Avoidance using a Drone by Alexander C. woods and Hung M. La (https://link.springer.com/chapter/10.1007/978-3-319-27857-5_76)
5. K. McGuire, G. de Croon, C. De Wagter, K. Tuyls and H. Kappen, "Efficient Optical Flow and Stereo Vision for Velocity Estimation and Obstacle Avoidance on an Autonomous Pocket Drone," in IEEE Robotics and Automation Letters, vol. 2, no. 2, pp. 1070-1076, April 2017, doi: 10.1109/LRA.2017.2658940.
6. A Survey on Unmanned Aerial Vehicle Collision Avoidance Systems by Hung Pham (1), Scott A. Smolka (1), Scott D. Stoller (1), Dung Phan (1), Junxing Yang (1).
7. Unmanned Aerial Vehicles (UAVs): Collision Avoidance Systems and Approaches Jawad N. Yasin, Sherif A.S. Mohamed, Mohammad-hashem Haghbayan, Jukka Heikkonen, Hannu Tenhunen, and Juha Plosila1 (Member, IEEE).
8. A. Devos, E. Ebeid and P. Manoonpong, "Development of Autonomous Drones for Adaptive Obstacle Avoidance in Real World Environments," 2018 21st Euromicro Conference on Digital System Design (DSD), 2018, pp. 707-710, doi: 10.1109/DSD.2018.00009.
9. Vision based drone obstacle avoidance by deep reinforcement learning by Zhihan Xue and Tad Gonsalves.
10. Obstacle avoidance drone by deep reinforcement learning and its racing with human pilot by Sang-Yun Shin, Yong-won Kang and Yong-Guk Kim.(<https://www.mdpi.com/597990>).
11. P. Chakravarty, K. Kelchtermans, T. Roussel, S. Wellens, T. Tuytelaars and L. Van Eycken, "CNN-based single image obstacle avoidance on a quadrotor," 2017 IEEE International Conference on Robotics and Automation (ICRA), 2017, pp. 6369-6374, doi: 10.1109/ICRA.2017.7989752.
12. S. Suherman, R. A. Putra and M. Pinem, "Ultrasonic Sensor Assessment for Obstacle Avoidance in Quadcopter-based Drone System," 2020 3rd International Conference on Mechanical, Electronics, Computer, and Industrial Technology (MECnIT), 2020, pp. 50-53, doi: 10.1109/MECnIT48290.2020.9166607.
13. Automatic quadcopter control avoiding obstacle using camera with integrated ultrasonic sensor by Hanafi Anis, Ahmad Haris Indra Fadhillah, Surya Darma, and Santoso Soekirno, published under licence by IOP Publishing Ltd. Hanafi Anis *et al* 2018 *J. Phys.: Conf. Ser.* 1011 012046.
14. Simple Four- Rotor obstacle avoidance design based on ultrasonic sensor by Yu Yang and Min Xu. Yu Yang and Min Xu 2018 *IOP Conf. Ser.: Earth Environ. Sci.* **170** 042112.
15. Yasin, J.N., Mohamed, S.A.S., Haghbayan, M.H. *et al.* Low-cost ultrasonic based object detection and collision avoidance method for autonomous robots. *Int. j. inf. technol.* **13**, 97–107 (2021). <https://doi.org/10.1007/s41870-020-00513-w>.

16. B. N. Sree, C. V. Raj and R. Madhavan, "Obstacle avoidance for UAVs used in road accident monitoring," 2017 1st International Conference on Electronics, Materials Engineering and Nano-Technology (IEMENTech), 2017, pp. 1-6, doi: 10.1109/IEMENTECH.2017.8077022.
17. <https://create.arduino.cc/projecthub/robocircuits/collision-avoidance-system-for-drones-7fed1>.
18. Obstacle Avoidance for Quadcopter using Ultrasonic Sensor by Muhammad Fazlur Rahman and Rianto Adhy Sasongko 2018 J. Phys.: Conf. Ser. 1005 012037. <https://iopscience.iop.org/article/10.1088/1742-6596/1005/1/012037/pdf>.
19. Borenstein, J. and Koren, Y. Obstacle Avoidance with Ultrasonic Sensors, IEEE J., 1998, pp.213-218.
20. Rahman, M.F., Wisnu, Sasongko, R.A. Obstacle's Contour Detection using Ultrasonic Sensors. Advance in Aerospace Science and Technology in Indonesia, Vol. 1.
21. https://www.researchgate.net/publication/325017457_Obstacle_Avoidance_for_Quadcopter_using_Ultrasonic_Sensor.

APPENDIX

Appendix A: Raw code for obstacle avoidance

```
#define SONAR_NUM 4 -----define total number of Ultrasonic Sensors, in our case
we are using 4 sensors
#define MAX_DISTANCE 200 -----define the maximum distance, this is an optional
definition because our project uses the minimum distance rather than maximum distance
#include <NewPing.h> -----Install the New Ping Library which is basically a file
which extracts distances from unlimited connected Ultrasonic Distance Sensors with Arduino
UNO

int d1, d2, d3, d4, ch1, ch2; -----define 4 variables which store distance from 4 Ultrasonic
Sensors & ch1 & ch2 stores data from Transmitter Channel 1 & 2 (Roll & Pitch)

NewPing sonar[SONAR_NUM] = -----defining TRIG PIN & ECHO PIN connections
{
  NewPing (4, 5, MAX_DISTANCE), -----one Ultrasonic sensor's TRIG & ECHO PINS
are connected to Arduino Pin 4 & 5 respectively
  NewPing (6, 7, MAX_DISTANCE), -----one Ultrasonic sensor's TRIG & ECHO PINS
are connected to Arduino Pin 6 & 7 respectively
  NewPing (8, 9, MAX_DISTANCE), -----one Ultrasonic sensor's TRIG & ECHO PINS
are connected to Arduino Pin 8 & 9 respectively
  NewPing (10, 11, MAX_DISTANCE) -----one Ultrasonic sensor's TRIG & ECHO
PINS are connected to Arduino Pin 10 & 11 respectively
};
void setup () -----this portion defines what Pins on Arduino will be used as input &
which as output
{
  Serial.begin(115200); ----- at what rate should data be read & processed (bytes/second)
[115200 is called baud rate]
  pinMode (2, INPUT); ----- Arduino Pin 2 will be used as input [Data will be taken
from Channel 1 of receiver]
  pinMode (3, INPUT); ----- Arduino Pin 3 will be used as input [Data will be taken
from Channel 2 of receiver]
  pinMode (1, OUTPUT); ----- Arduino Pin 1 will be used as output [Data will be
sent to Channel 1 of KK Board]
  pinMode (12, OUTPUT); ----- Arduino Pin 12 will be used as output
[Data will be sent to Channel 2 of KK Board]
}

void loop ()
{
  d1 = sonar [0]. ping_cm (); -----calculate distance from 1st sensor and store to
variable d1
  d2 = sonar [1]. ping_cm (); -----calculate distance from 2nd sensor and store
to variable d2
  d3 = sonar [2]. ping_cm (); -----calculate distance from 3rd sensor and store
to variable d3
  d4 = sonar [3]. ping_cm (); -----calculate distance from 4th sensor and store
to variable d4
```

```

ch1 = pulseIn (2, HIGH, 25000); -----start taking data from Receiver channel 1
(Arduino Pin 2) & store to variable ch1
ch2 = pulseIn (3, HIGH, 25000); -----start taking data from Receiver channel 2
(Arduino Pin 3) & store to variable ch2

```

```

mod_ch1 = map (ch1, 1000, 2000, -500, 500); -----adjust whatever value is
received from channel 1 of receiver between -500 & 500
mod_ch2 = map (ch2, 1000, 2000, -500, 500); -----adjust whatever value is
received from channel 2 of receiver between -500 & 500

```

```

if(d1<30) -----if distance on first sensor is less than 30 cm
{
  mod_ch2 = (d1/30) *(-500); -----take the drone away from obstacle
automatically by making the drone go in opposite direction
  analog Write (1, mod_ch2);
}

```

```

if(d2<30)
{
  mod_ch2 = (d2/30) *(500); -----take the drone away from obstacle
automatically by making the drone go in opposite direction
  analog Write (1, mod_ch2);
}

```

```

if(d3<30)
{
  mod_ch1 = (d3/30) *(500); -----take the drone away from obstacle
automatically by making the drone go in opposite direction
  analog Write (1, mod_ch2);
}

```

```

if(d4<30)
{
  mod_ch2 = (d4/30) *(-500); -----take the drone away from obstacle
automatically by making the drone go in opposite direction
  analog Write (1, mod_ch2);
}
}

```

Appendix B: Implemented code for obstacle avoidance

```

#include <NewPing.h>
#define SONAR_NUM 4
#define MAX_DISTANCE 200

//connect 4 HCSR04 on (4,5)[left], (6,7)[right], (8,9)[forward], (10, 11)[backward];

NewPing sonar[SONAR_NUM] =
{
  NewPing(4, 5, MAX_DISTANCE),

```

```
NewPing(6, 7, MAX_DISTANCE),
NewPing(8, 9, MAX_DISTANCE),
NewPing(10, 11, MAX_DISTANCE),
};
```

```
int trigger;
int roll;
int pitch;
int roll_to_kk;
int pitch_to_kk;
int obstacle_left;
int obstacle_right;
int obstacle_forward;
int obstacle_backward;
```

sensitivity = 4; //Drone Response Sensitivity can be changed from here, large number will result in lower sensitivity & vice versa

```
void setup()
{
  Serial.begin(9600);
  pinMode(14, INPUT); //connect Channel 5 from reciever to arduino Pin 14 (analog A0)
  pinMode(2, INPUT); //connect roll channel from reciever to Arduino Pin 2
  pinMode(3, INPUT); //connect pitch channel from reciever to Arduino Pin 3
  pinMode(12, OUTPUT); //connect Arduino Pin 12 to roll channel of KK Board
  pinMode(13, OUTPUT); //connect Arduino Pin 13 to pitch channel of KK Board
}
```

```
void loop()
{
  trigger = pulseIn(14, HIGH);
  roll = pulseIn(2, HIGH);
  pitch = pulseIn(3, HIGH);
  obstacle_left = int(sonar[0].ping_cm());
  obstacle_right = int(sonar[1].ping_cm());
  obstacle_forward = int(sonar[2].ping_cm());
  obstacle_backward = int(sonar[3].ping_cm());
```

```
if (trigger>500)
{
```

```
  if(obstacle_left < 50)
  {
    roll_to_kk = roll + int(roll/4)
    analogWrite(12, roll_to_kk);
  }
```

```
  if(obstacle_right < 50)
  {
    roll_to_kk = roll - int(roll/4)
```

```

analogWrite(12, roll_to_kk);
}

if(obstacle_forward < 50)
{
pitch_to_kk = pitch - int(pitch/4)
analogWrite(13, pitch_to_kk);
}

if(obstacle_backward < 50)
{
pitch_to_kk = pitch + int(pitch/4)
analogWrite(13, pitch_to_kk);
}

}

}

```

Appendix C: Future Work and Alternative Recommendations

As a part of alternative future work, we came up with three different approach for the drone obstacle avoidance, which consists of avoiding the drone with using LIDAR sensors, using infrared sensors, and with using event cameras.

Appendix D: Matlab model for obstacle avoidance

This submission contains the implementation of optical flow algorithm for obstacle avoidance. Here, the drone to traverses through a Simulink® 3D Animation arena using optical flow algorithm calculated using the drone's front view camera. This model aims at helping you to get started to use computer vision along with control systems in your ground/aerial robots. Using this model, you can get your UAV/UGV to avoid obstacles using the front-facing (bird's eye view) vision data of the vehicle.

Assumptions in the model

- While designing the system, the drone is considered as a point mass and the drone dynamics are not considered.
- The feedback from the drone is taken from the Simulink 3D Animation sensors.
- The model assumes that the user knows at what point the controller switches from using optical flow to using PID controller.
- The model also assumes that the x-coordinate landing location is greater than the x-coordinate of the point of transition from optical flow controller to PID Controller.